

UNIWERSYTET MARII CURIE-SKŁODOWSKIEJ
W LUBLINIE
Wydział Matematyki, Fizyki i Informatyki

Kierunek: Informatyka

Dominik Zaręba
Nr albumu: 318765

Aplikacja webowa do analizowania sygnałów EEG z wykorzystaniem modelu uczenia maszynowego *(Machine learning–based web application for EEG signal analysis)*

Praca licencjacka
napisana w Katedrze Neuroinformatyki i Inżynierii Biomedycznej
pod kierunkiem dr. inż. Piotra Schneidera

LUBLIN ROK 2026

Streszczenie

Celem pracy było stworzenie aplikacji webowej umożliwiającej analizę sygnałów rejestrowanych metodą elektroencefalografii (ang. electroencephalography, EEG), a także ich klasyfikację do siedmiu klas emocjonalnych. Do przygotowania i oceny rozwiązania wykorzystano zbiór danych SEED-VII. Zaimplementowany proces przetwarzania obejmował segmentację i filtrację sygnału, obliczanie entropii różniczkowej, wygładzanie uzyskanych cech za pomocą liniowego systemu dynamicznego oraz przygotowanie wektora przekazywanego do klasyfikatora.

W przeprowadzonym badaniu porównano maszynę wektorów nośnych (ang. support vector machine, SVM), las losowy (ang. random forest) oraz ekstremalne wzmacnianie gradientowe (ang. extreme gradient boosting, XGBoost). Modele oceniono przy użyciu trzech różnych schematów walidacji: zależnej od uczestnika (ang. subject-dependent, SD), z wyłączeniem jednego uczestnika (ang. leave-one-subject-out, LOSO) oraz niezależnej od bodźców (ang. stimulus-independent, SI). Najwyższą trafność uzyskał model SVM wykorzystujący cechy własne w walidacji zależnej od uczestnika - 42,06 %. W bardziej wymagającej walidacji LOSO, lepiej odpowiadającej sytuacji, w której z aplikacji korzysta nowa osoba, najlepszy okazał się model XGBoost oparty na cechach własnych. Osiągnął on 24,25 % trafności i 0,2371 macro-F1. Różnica między wynikami poszczególnych walidacji wskazuje, że rozpoznawanie emocji u osób nieobecnych w zbiorze treningowym pozostaje trudnym zadaniem. Analiza wartości probabilistycznych wykazała również, że pewność predykcji modeli nie zawsze odpowiadała ich rzeczywistej skuteczności.

Aplikacja pozwala wczytywać dane EEG i gotowe cechy, wykonywać klasyfikację, przeglądać sygnał oraz eksportować wyniki analizy. Testy automatyczne i ręczne potwierdziły poprawność działania jej najważniejszych funkcji. Mimo że uzyskana skuteczność nie pozwala jeszcze na praktyczne wykorzystanie systemu, przygotowana aplikacja stanowi kompletny prototyp badawczy i może być podstawą do dalszego rozwijania metod rozpoznawania emocji na podstawie sygnałów EEG.

Słowa kluczowe: elektroencefalografia, EEG, rozpoznawanie emocji, uczenie maszynowe, porównanie modeli, przetwarzanie sygnałów, aplikacja webowa, SEED-VII

Abstract

The purpose of this thesis was to develop a web application for analysing signals recorded using electroencephalography (EEG) and classifying them into seven emotion classes. The SEED-VII dataset was used to develop and evaluate the proposed solution. The implemented processing pipeline included signal segmentation and filtering, differential entropy calculation, smoothing of the extracted features using a linear dynamical system, and preparation of a feature vector supplied to the classifier.

The study compared a support vector machine (SVM), a random forest, and extreme gradient boosting (XGBoost). The models were evaluated using three validation schemes: subject-dependent (SD), leave- one-subject-out (LOSO), and stimulus-independent (SI) validation. The highest accuracy, 42.06 %, was achieved by the SVM model using custom features in subject-dependent validation. In the more demanding LOSO setting, which more closely reflects a scenario in which the application is used by a new person, the best result was achieved by XGBoost using custom features. It achieved an accuracy of 24.25 % and a macro-F1 score of 0.2371. The differences between the validation results indicate that recognising emotions for individuals who were not represented in the training dataset remains a challenging task. The analysis of the probabilistic outputs also showed that the confidence of the models' predictions did not always correspond to their actual performance.

The application can load EEG preprocessed data and previously extracted features, classify the input as one of seven emotion classes, visualise the signal, and export the analysis results. Automated and manual tests verified that all crucial functions worked correctly. Although the achieved performance is not yet sufficient for practical use, the resulting application provides a complete research prototype and may serve as a foundation for the further development of EEG- based emotion recognition methods.

Keywords: electroencephalography, EEG, emotion recognition, machine learning, model comparison, signal processing, web application, SEED-VII

Oświadczenie

Oświadczam, że niniejsza praca została wykonana samodzielnie, a wszystkie wykorzystane źródła zostały odpowiednio zacytowane.

Dominik Zaręba

Spis treści

1 Wstęp **1**
1.1 Cel i zakres pracy . . . 1
1.2 Motywacja . . . 2
1.3 Struktura pracy . . . 2

2 Podstawy teoretyczne **3**
2.1 Wprowadzenie historyczne . . . 3
2.2 Fale mózgowe i częstotliwości . . . 3
2.3 Fizjologia powstania sygnału . . . 4
2.4 Fizyczne ograniczenia i charakterystyka sygnału EEG . . . 5
2.5 Pomiar i postać cyfrowa sygnału . . . 6
2.5.1 Elementy sprzętu badawczego . . . 7
2.6 Przetwarzanie sygnału . . . 9
2.6.1 Przetwarzanie wstępne . . . 9
2.6.1.1 Import danych oraz konfigurowanie lokalizacji elektrod . 9
2.6.1.2 Filtrowanie cyfrowe . . . 9
2.6.1.3 Re-referencja . . . 11
2.6.1.4 Segmentacja sygnału . . . 11
2.6.1.5 Korekcja linii bazowej . . . 12
2.6.1.6 Identyfikowanie uszkodzonych kanałów i ich interpolacja 12
2.6.1.7 Odrzucanie próbek . . . 12
2.6.1.8 Analiza składowych niezależnych (ang. independent component analysis, ICA) . . . 12
2.6.2 Ekstrakcja cech . . . 13
2.7 Modele uczenia maszynowego . . . 15
2.8 Środowisko i narzędzia . . . 16

3 Metodyka i implementacja systemu **17**
3.1 Wprowadzenie . . . 17
3.2 Cel projektu, wymagania oraz ograniczenia . . . 17
3.2.1 Cel techniczny systemu . . . 17
3.2.2 Wymagania funkcjonalne . . . 18
3.2.3 Wymagania niefunkcjonalne . . . 18
3.2.4 Ograniczenia wejścia . . . 19
3.3 Zbiór danych. Przygotowanie i charakterystyka . . . 20
3.3.1 Źródło danych . . . 20
3.3.2 Próbki danych i etykiety . . . 21
3.3.3 Struktura plików . . . 21
3.3.4 Warianty zbioru treningowego . . . 22
3.4 Przetwarzanie i ekstrakcja cech . . . 23
3.4.1 Ogólny przebieg przetwarzania . . . 23
3.4.2 Segmentacja i filtracja pasmowa . . . 24
3.4.3 Entropia różniczkowa (DE) . . . 25
3.4.4 Wygładzanie LDS . . . 26

3.4.5 Agregacja do stałego wymiaru . . . 26
3.5 Modele uczenia maszynowego . . . 27
3.5.1 Walidacje modeli uczenia maszynowego . . . 27
3.5.2 Metryki . . . 29
3.5.3 Punkty odniesienia . . . 33
3.5.4 Wybór emocji przez aplikację i walidację . . . 34
3.5.5 Wybór i zapis modelu . . . 35
3.6 Architektura i implementacja aplikacji webowej . . . 35
3.6.1 Wczytywanie i walidacja pliku . . . 38
3.6.2 Dobór właściwego modelu . . . 39
3.6.3 Przebieg interakcji użytkownika . . . 40

4 Wyniki i analiza **43**
4.1 Przygotowanie analizy . . . 43
4.2 Zestawienie i porównanie klasyfikatorów oraz modeli bazowych . . . 44
4.3 Macierz pomyłek . . . 46
4.4 Wiarygodność wyników . . . 48
4.5 Porównanie literaturowe modeli . . . 49
4.6 Weryfikacja aplikacji . . . 51
4.6.1 Testy automatyczne . . . 51
4.6.2 Ręczne testy . . . 53

5 Podsumowanie i wnioski **57**
5.1 Ocena realizacji celu i rezultatów pracy . . . 57
5.2 Ograniczenia . . . 58
5.3 Kierunki dalszych prac . . . 58

Bibliografia **67**

1. Wstęp

Ostatnie lata charakteryzują się dynamicznym rozwojem technologicznym. Przedmioty codziennego użytku oraz usługi, które dotychczas wymagały od nas fizycznej ingerencji, przechodzą szybką cyfryzację. Z kolei uczenie maszynowe powoli przełamuje monopol wielkich korporacji z branży technologii informacyjnych (ang. information technology, IT) na rzecz mniejszych zespołów programistycznych, które z jego pomocą potrafią tworzyć funkcjonalne aplikacje, urządzenia oraz rozwijać innowacyjne technologie [33, 34, 35]. Do mediów głównego nurtu coraz częściej trafia również temat interfejsu mózg-komputer (ang. brain-computer interface, BCI). Dzieje się to głównie za sprawą głośnych projektów, takich jak "Telepathy", czy "Blindsight" firmy Neuralink [51, 52]. Dzięki utrwalanym standardom w dziedzinach kognitywistyki, neurobiologii i inżynierii biomedycznej, obszar ten staje się jednym z najważniejszych kierunków rozwoju technologii.

Doskonałym przykładem takiego standardu jest elektroencefalografia (ang. electroencephalography, EEG). Choć jej początki sięgają przełomu XIX i XX wieku, to po dziś dzień jest nieprzerwanie wykorzystywana w wielu projektach, badaniach oraz terapiach. Ta nieinwazyjna metoda pomiaru aktywności elektrycznej mózgu udowadnia, że technologia stosowana od ponad stu lat wciąż potrafi zaskakiwać nowymi możliwościami [1]. Elektroencefalografia stale pomaga badaczom w poszerzaniu wiedzy oraz wspieraniu diagnostyki takich schorzeń jak choroba Alzheimera, zespół nadpobudliwości psychoruchowej z deficytem uwagi (ang. attention-deficit/hyperactivity disorder, ADHD) czy padaczka [19, 20, 21].

Łącząc dobrze znany standard, jakim jest EEG, z nowoczesnymi modelami uczenia maszynowego, zyskujemy potężne narzędzie do analizy sygnału w sposób, który wcześniej wydawał się niemożliwy. Zakres możliwości jakie dają nam te technologie jest niezwykle szeroki. Zaczynając od prostych obserwacji stanu czuwania, przez badania stanów emocjonalnych, aż po diagnozę dolegliwości [10, 22, 23]. Nawet niewielkie anomalie czy niezauważalne dla człowieka wzorce badań mogą wkrótce stać się ogromnym krokiem naprzód w diagnostyce wielu chorób.

1.1 Cel i zakres pracy

Celem niniejszej pracy jest zaprojektowanie oraz implementacja aplikacji webowej do analizy sygnałów EEG z wykorzystaniem modelu uczenia maszynowego.

Aplikacja skupia się na klasyfikacji stanów emocjonalnych na podstawie danych EEG, zgodnych ze strukturą i specyfikacją zbioru danych SEED-VII. W założeniu projekt ma charakter badawczy i ma ukazać, w jaki sposób integracja różnych technologii pozwala na stworzenie funkcjonalnego prototypu. Rozwój narzędzia przy dysponowaniu odpowiednimi zasobami mógłby stanowić podstawę dla pełnoprawnych rozwiązań medycznych

bądź funkcjonalnego narzędzia analitycznego, statystycznego, czy agregującego. Część eksperymentalna ma na celu porównanie modeli uczenia maszynowego: maszyny wektorów nośnych (ang. support vector machine, SVM), lasu losowego (ang. random forest, RF) oraz ekstremalnego wzmacniania gradientowego (ang. extreme gradient boosting, XGBoost), przedstawiając ich możliwości w testach. Implementacja aplikacji jest poprzedzona rozdziałem teoretycznym.

1.2 Motywacja

Do wyboru niniejszego tematu przyczyniły się dynamicznie rozwijające się trendy w sektorze IT oraz personalne zainteresowanie autora kognitywistyką.

Wyraźnie rosnące znaczenie uczenia maszynowego w wielu dziedzinach nauki zachęca do znalezienia nowych możliwości jego integracji z inżynierią biomedyczną. Rozwój technologii takich jak uczenie maszynowe, BCI czy analiza EEG to obecnie jeden z najbardziej perspektywicznych kierunków rozwoju oprogramowania - warto się nim zainteresować jak najwcześniej [36]. Motywacją do podjęcia tematu była również możliwość zastosowania metod przetwarzania sygnałów EEG i uczenia maszynowego w nowoczesnej, intuicyjnej i łatwo dostępnej aplikacji webowej. Istotnym aspektem badawczym była ocena skuteczności klasyfikacji etykiet emocjonalnych.

1.3 Struktura pracy

Praca obejmuje omówienie teoretycznych podstaw elektroencefalografii - od fizjologii powstania sygnału, przez metody jego pomiaru, aż po przetwarzanie i wykorzystanie w praktyce. Wiedza ta jest niezbędna do właściwego zrozumienia charakteru danych, na których autor pracował podczas tworzenia aplikacji.

W kolejnych rozdziałach przedstawiono proces implementacji, scharakteryzowano wykorzystany zbiór danych oraz opisano wymagania i ograniczenia wynikające m.in. ze specyfiki projektu badawczego. W dalszej części pracy zaprezentowano testy, wyniki działania modeli oraz płynące z nich wnioski. Całość zamyka podsumowanie zrealizowanej pracy oraz omówienie propozycji i perspektyw rozwoju systemu w jego kolejnych iteracjach.

2. Podstawy teoretyczne

2.1 Wprowadzenie historyczne

Osobą, która stoi za odkryciem fal elektrycznych w mózgu jest Richard Caton. To, co początkowo zaobserwował w 1875 roku w mózgach królików i małp było punktem wyjścia do rozwoju technologii, która zrewolucjonizowała podejście naukowców do kognitywistyki, a co za tym idzie neurobiologii, informatyki, psychologii i wielu innych dziedzin nauki. W 1924 r. niemiecki naukowiec Hans Berger dokonał pierwszych pomiarów EEG, których wyniki opublikowano 5 lat później. Berger w swoich wczesnych protokołach pisał o wykorzystaniu galwanometru strunowego do badania fal mózgowych na pacjentach ze schorzeniami mózgu. Już wtedy badał zapisy EEG chorych na padaczkę i identyfikował artefakty związane z mruganiem, czy ruchami twarzy. Jego badania skupiały się także na zmianach w zapisie sygnału osób zdrowych w różnych stanach świadomości [1].

W 1934 roku opisano regularne oscylacje sygnału EEG o częstotliwości około 10–12 Hz, określane jako rytm alfa [1, 5]. Podawany zakres częstotliwości różni się nieco w zależności od źródła.

2.2 Fale mózgowe i częstotliwości

Pasma częstotliwości służą do opisu składowych widmowych sygnału EEG. Ich granice mogą różnić się między publikacjami. Zakresy przyjęte w niniejszej pracy, zgodne z implementacją, zestawiono w tabeli 2.1.

Tabela 2.1. Pasma częstotliwości sygnału EEG [1, 4, 5]

Pasmo	Zakres częstotliwości [Hz]
Delta	1-4
Theta	4-8
Alfa	8-14
Beta	14-31
Gamma	31-50

W związku ze wspomnianymi różnicami, podział aktywności EEG na pasma częstotliwościowe ma charakter umowny. Szczególnie widoczne jest to w przypadku rytmu alfa, którego częstotliwość szczytowa może zmieniać się wraz z obciążeniem poznawczym i wykazywać wyraźne różnice między badanymi. Z tego powodu sztywne przyjęcie jednego zakresu częstotliwości może prowadzić do pominięcia części aktywności charakterystycznej dla danej osoby [37]. Aktywności w określonym paśmie nie należy również traktować

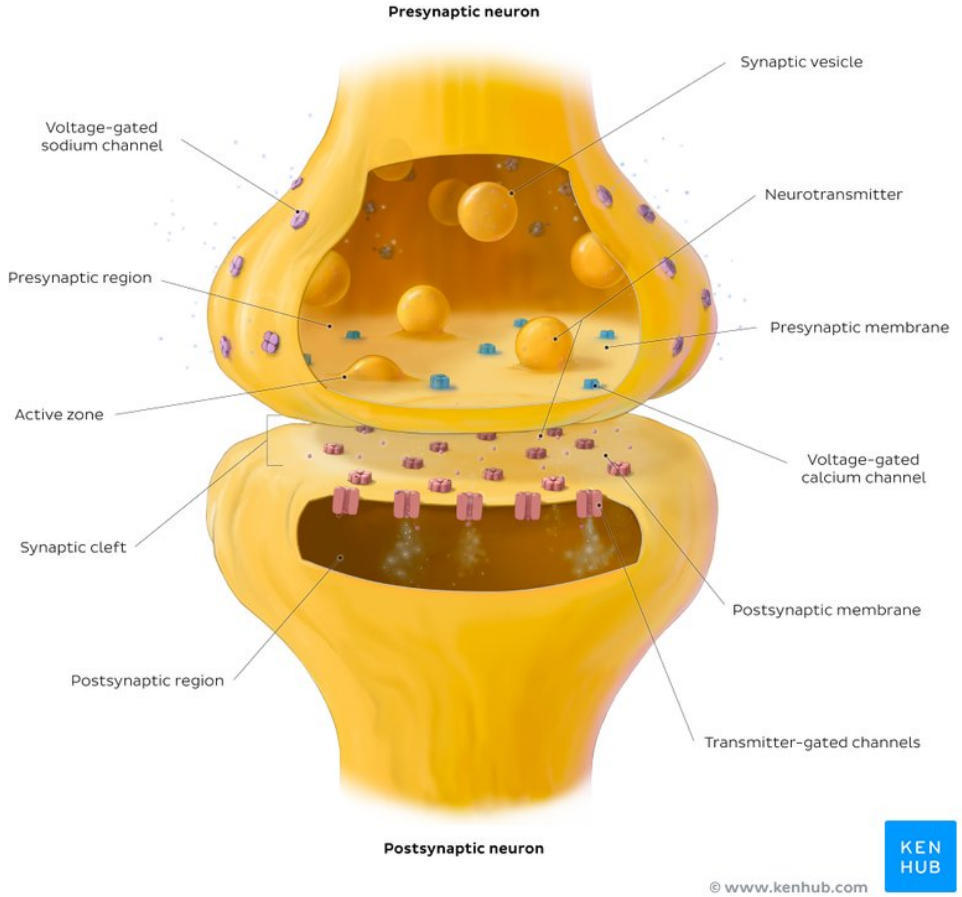
jako jednoznacznego wskaźnika pojedynczego stanu psychofizjologicznego. Dla przykładu - badania z wykorzystaniem elektrokortykografii wykazały, że zamknięcie oczu powoduje nie tylko wzrost aktywności alfa, szczególnie w obszarach potylicznych, lecz także zmiany mocy w pasmach delta, theta i beta. Stan badanego może więc wpływać jednocześnie na kilka zakresów częstotliwości [38]. Szczególnej ostrożności wymaga interpretacja wyższych częstotliwości rejestrowanych z powierzchni głowy. Zakresy beta i gamma częściowo pokrywają się z aktywnością mięśniową. W badaniu porównującym zapis przed blokadą nerwowo-mięśniową i podczas niej zaobserwowano bardzo duże różnice mocy dla częstotliwości powyżej 20 Hz. Wyniki te wskazują, że analiza wyższych pasm wymaga skutecznego wykrywania i usuwania artefaktów mięśniowych [39]. Pasma częstotliwości stanowią użyteczny punkt odniesienia, jednak ich interpretacja często wymaga ostrożności. Obserwowana aktywność w poszczególnych pasmach zależy więc nie tylko od procesów neuronalnych, lecz również od wspomnianej obecności szumów i artefaktów oraz zastosowanych metod filtracji. Na wiarygodność wyznaczanej mocy pasm i ich późniejszą interpretację wpływa między innymi proces wstępnego przetwarzania danych, opisany w podrozdziale 2.6.1.

2.3 Fizjologia powstania sygnału

Największy, dominujący wpływ na zapis EEG ma aktywność elektryczna kory mózgowej - wynika to bezpośrednio z jej powierzchniowego położenia najbliżej elektrod pomiarowych [1, 18].

Rejestrowana aktywność elektryczna pochodzi głównie z pobudzenia synaptycznego dendrytów neuronów piramidowych zlokalizowanych w korze mózgowej. Pobudzenie synaptyczne, określane w fizjologii jako postsynaptyczny potencjał pobudzający (ang. excitatory postsynaptic potential, EPSP), czyli tymczasowej zmiany polaryzacji błony neuronu (różnicy potencjałów ładunków elektrycznych po obu stronach błony komórkowej). Po dotarciu impulsu nerwowego do zakończenia presynaptycznego uwalniane są neuroprzekaźniki, co powoduje otwarcie kanałów jonowych w błonie neuronu postsynaptycznego. Przepływ jonów przez te kanały powoduje depolaryzację, czyli zmniejszenie ładunku ujemnego wewnątrz neuronu. Pojedynczy neuron nie byłby jednak w stanie wygenerować na tyle silnego sygnału, aby elektroda mogła zarejestrować napięcie elektryczne. Rejestrowany na powierzchni głowy sygnał EEG wynika z sumowania zsynchronizowanej aktywności dużych populacji neuronów piramidowych. [1, 5, 18].

Istotny udział w procesie powstawania sygnału ma również postsynaptyczny potencjał hamujący (ang. inhibitory postsynaptic potential, IPSP). W porównaniu do EPSP, w tym procesie hamowanie synaptyczne zmniejsza prawdopodobieństwo powstania potencjału czynnościowego. Jednym z jego mechanizmów jest hiperpolaryzacja, podczas której potencjał wnętrza komórki staje się bardziej ujemny względem otoczenia [18].



Rysunek 2.1. Mechanizm przekazywania sygnału na poziomie pojedynczej synapsy [11]

Na rysunku 2.1 przedstawiono mechanizm przekazywania sygnału na poziomie pojedynczej synapsy obrazujący opis fizjologicznego powstania sygnału. Z pęcherzyków synaptycznych (synaptic vesicle) widocznych na górze schematu w obrębie neuronu presynaptycznego uwalniają się żółte cząsteczki neuroprzekaźnika (neurotransmitter). Uwolnione z nich neuroprzekaźniki docierają do dolnej części (błony postsynaptycznej), wiążąc się z receptorami, co skutkuje otwarciem różowych struktur opisanych na schemacie jako kanały bramkowane przekaźnikiem (transmitter-gated channels). Schemat pokazuje również "strumień" jonów napływający przez otwarte różowe kanały do wnętrza dolnego neuronu postsynaptycznego - właśnie ten napływ jonów zmienia polaryzację błony, generując potencjał pobudzający.

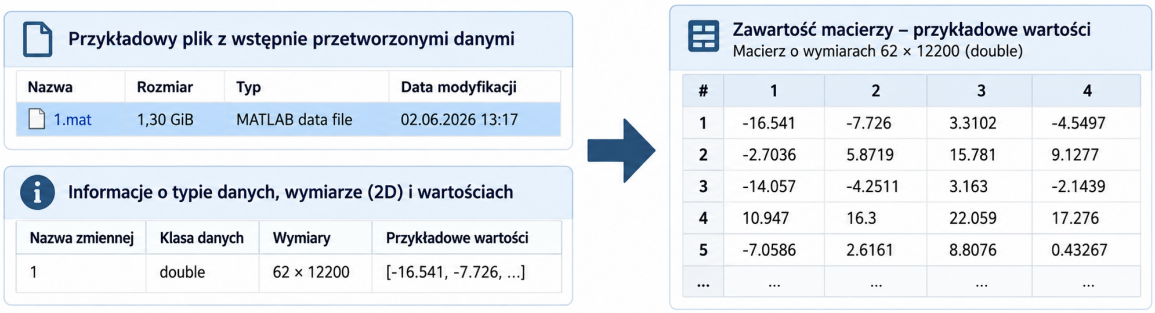
2.4 Fizyczne ograniczenia i charakterystyka sygnału EEG

Z punktu widzenia przetwarzania danych i uczenia maszynowego, czyli najważniejszego w kontekście niniejszej pracy, sygnał elektroencefalograficzny można określić jako wielokanałowy szereg czasowy, czyli strukturę danych, którą należy rozumieć następująco:

- szereg czasowy to sekwencja danych pomiarowych, które uporządkowane są chronologicznie. Kolejne odczyty rejestrowane są w równych odstępach czasu wynikających z częstotliwości próbkowania - np. 200 Hz, oznaczające odczyt co 5 milisekund.
- wielokanałowość oznacza, że wiele szeregów czasowych jest rejestrowanych jednocześnie z różnych elektrod rozmieszczonych na głowie w trakcie pomiaru.

Powstałe dane wejściowe z perspektywy inżynierii systemów informatycznych posiadają pewne fizyczne ograniczenia:

- Niska amplituda - napięcie mierzone na powierzchni czaszki jest bardzo małe, waha się zazwyczaj w przedziale od 0,5 do 100 µV. Sygnał wymaga więc silnego wzmocnienia i jest skrajnie podatny na wszelkiego rodzaju zakłócenia [1, 4].
- Szumy i zakłócenia - właściwości przewodzące tkanek między mózgiem, a elektrodami, takich jak opony mózgowe, kości i skóra, wpływają na amplitudę rejestrowanego sygnału. Dodatkowe zakłócenia powodują mruganie oczu i praca mięśni. W rezultacie surowy sygnał EEG ma niewielką amplitudę i może zawierać szumy oraz artefakty utrudniające jego interpretację [3, 18].
- Zmiany w czasie - sygnał EEG ulega ciągłym zmianom w czasie, co może znacząco utrudniać pracę nad danymi treningowymi do modelu uczenia maszynowego, testowymi, czy także ich analizą [5, 6].



Rysunek 2.2. Przykładowe dane EEG [10, 43]

Na rysunku 2.2 przedstawiony został przykładowy plik z danymi wstępnie przetworzonymi. Dane zostały pozyskane z SEED-VII, wielomodalnego zbioru danych przeznaczonego do rozpoznawania emocji, opracowanego przez Uniwersytet Jiao Tong w Szanghaju (ang. Shanghai Jiao Tong University, SJTU). Dane udostępniono autorowi do celów naukowych. Tablica zawiera 62 kanały po 12 200 próbek, co odpowiada 61 s przy próbkowaniu 200 Hz [10].

2.5 Pomiar i postać cyfrowa sygnału

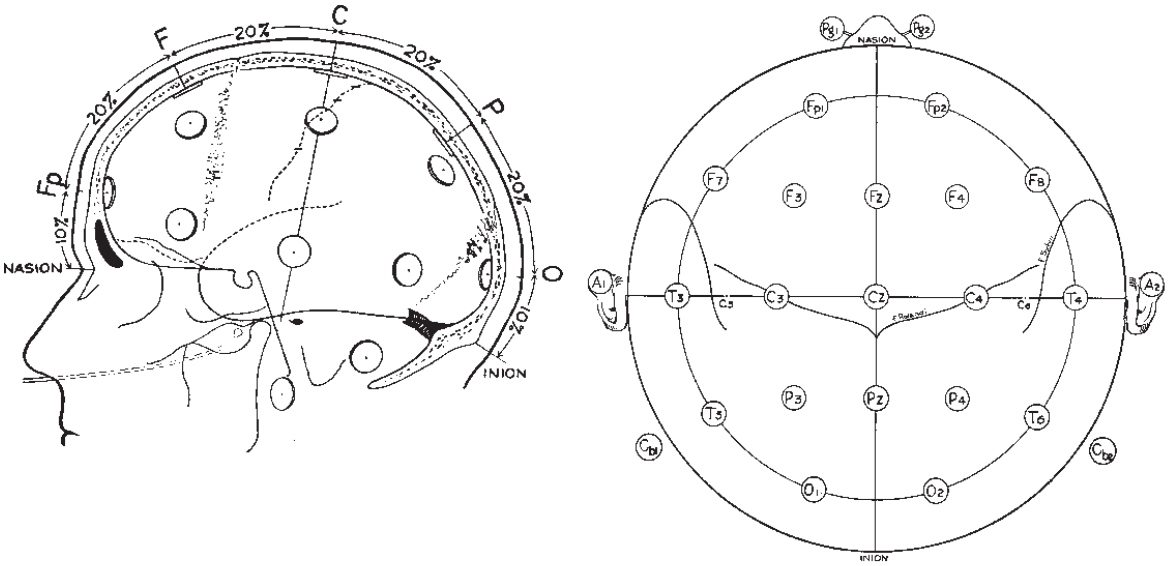
Aby uzyskać pomiar sygnału potrzebny jest odpowiedni sprzęt. Jego dobór wśród przeprowadzanych przez naukowców badań jest dość elastyczny i może wpływać na różnice w wartościach pomiaru [1, 3, 24].

2.5.1 Elementy sprzętu badawczego

Kluczowymi elementami przykładowego zestawu są:

- Elektrody - jednym z najważniejszych zagadnień dotyczących elektrod jest system ich rozmieszczenia. Na przestrzeni lat powstało kilka ustandaryzowanych systemów, aby ułatwić działanie na danych, zachować powtarzalność wyników znacząco ułatwiając analizę.

  Taki standard to między innymi "10-20" (oznaczający procentowe odległości między elektrodami w odniesieniu do konkretnych miejsc orientacyjnych) [1].



Rysunek 2.3. Rozmieszczenie elektrod w systemie 10–20 [12]

Rysunek 2.3 przedstawia pomiar metodą 10-20. Pomiar rozciąga się od punktu na czaszce, który znajduje się u nasady nosa - znajduje się tam wyczuwalne wgłębienie pomiędzy oczami (nasion), aż po inion, punkt leżący po przeciwnej stronie zwany środkiem guzowatości potylicznej zewnętrznej - ten z kolei łatwo wyczuwalny w postaci zgrubienia na kości potylicznej. W tym przypadku to te punkty orientacyjne decydują o użyciu metody - odległości między sąsiednimi elektrodami to 10 % lub 20 % całkowitej długości mierzonej od nasion do inion [1, 12].

Oznaczenia lokalizacji elektrod zestawiono w tabeli 2.2.

Tabela 2.2. Oznaczenia lokalizacji elektrod [12]

Fp	Biegun czołowy (Frontal Pole)
F	płat czołowy (frontal)
C	okolica centralna
P	płat ciemieniowy (Parietal)
O	płat potyliczny (Occipital)

Najczęściej wyróżniane typy elektrod zestawiono w tabeli 2.3 [1, 5].

Tabela 2.3. Rodzaje elektrod [1]

1	Elektrody jednorazowe
2	Elektrody miseczkowe wielokrotnego użytku
3	Elektrody typu headband (zazwyczaj będące pełnym, funkcjonalnym urządzeniem)
4	Elektrody igłowe

- Czepek

  Czepek jest elementem zestawu badawczego, który służy do rozmieszczania elektrod - zazwyczaj według wcześniej opisanych standardów (10–20, 10-10 etc.). Z biegiem lat powstaje coraz więcej nowych technologii i metod, których używa się do modyfikacji czepków. Zazwyczaj do badań wykorzystywano żele przewodzące, które zapewniały stabilniejszy transfer sygnałów. Możemy wyróżnić także metodę badań "na sucho", która nie wymaga użycia substancji, czy chociażby czepek oparty na soli fizjologicznej [7].
- Wzmacniacz i przetwornik analogowo-cyfrowy (ang. analog-to-digital converter, ADC)

  Z racji na niskie wartości wejściowe surowych sygnałów EEG, przydatnym urządzeniem jest wzmacniacz. Jego podstawową rolą jest wzmocnienie małych różnic potencjałów [24].

  Według [1] wzmacniacz powinien mieć impedancję wejściową wynoszącą co najmniej 100 MΩ oraz współczynnik tłumienia sygnału wspólnego (ang. common-mode rejection ratio, CMRR) o wartości co najmniej 100 dB, co pomaga tłumić zakłócenia.

  Przetworniki analogowo-cyfrowe służą do zmiany sygnału analogowego na postać cyfrową, umożliwiając jego dalsze przetwarzanie, filtrowanie i analizę komputerową [24, 25, 26].

  Aby sygnał mógł zostać finalnie wykorzystany w analizie komputerowej musi przejść przez proces próbkowania oraz kwantyzacji.

  Próbkowanie (inaczej dyskretyzacja) to proces jaki musi przejść sygnał analogowy, aby ostatecznie otrzymać sygnał cyfrowy. Określa się konkretne chwile, w których wykonuje się pomiar wartości chwilowej sygnału ciągłego. W wyniku tej procedury otrzymujemy sygnał dyskretny. Częstotliwość próbkowania dobiera się według analizowanego pasma. W przypadku sygnału dolnopasmowego powinna przekraczać ona dwukrotność jego najwyższej częstotliwości [2, 3].

  W pracy [2] podano przykład ilustrujący twierdzenie Nyquista-Shannona o próbkowaniu: "Częstotliwość próbkowania plików audio wynosi często 44,1 kHz. To nieco więcej niż dwukrotność 20 kHz, które słyszą ludzie".

  Ostatecznie dochodzimy do kwantyzacji zmniejszającej precyzję sygnałów. Każdy przetwornik ma układ elektroniczny, zwany kwantyzatorem. Im większa liczba bitów kwan-

tyzatora, tym większa dokładność otrzymanego sygnału. Najnowsze systemy EEG posiadają przetworniki o rozdzielczości 24 bitów, co przekłada się na aż 16 777 216 poziomów napięcia [25, 26].

2.6 Przetwarzanie sygnału

Otrzymany sygnał cyfrowy po procesie kwantyzacji może być już przetwarzany przez urządzenia. Aby móc działać na danych należy przejść przez kilka kroków niwelujących pozostałe szumy, zakłócenia, czy artefakty. Jest to niezwykle ważne, gdyż niewłaściwe przetwarzanie może zniekształcić sygnał i wyniki analizy [3, 5].

Choć surowy sygnał może być używany jako dane treningowe we współczesnych modelach uczenia maszynowego, to należy wziąć pod uwagę wady i wyzwania, jakie stoją za zastosowaniem tego typu podejścia. W czasie badania nasze ciało jest w ciągłym, mimowolnym ruchu, co wiąże się z ruchem gałek ocznych, czy skurczami mięśni. Powoduje to powstanie artefaktów, które fałszują wynik końcowy [3, 5, 9, 17].

Opracowywana w pracy aplikacja jest ściśle związana z emocjami odczuwanymi przez człowieka w czasie stymulacji. Gdy nie wykluczymy wcześniej wspomnianych przeszkód z sygnału, może się okazać, że dla większości próbek model przewidzi błędne emocje, np. błędnie interpretując momenty skurczów mięśni jako moment odczuwania strachu.

2.6.1 Przetwarzanie wstępne

Standardowy krok w celu przygotowania użytecznych danych zaraz po ich uzyskaniu od osoby badanej to przetwarzanie wstępne (ang. preprocessing). Poniżej przedstawiono typowe operacje preprocessingu - nie wszystkie są wykonywane przez aplikację, lecz ich znajomość znacząco ułatwia zrozumienie funkcjonalności, przeznaczenia i genezy powstania danych ze zbiorów [5, 17, 27, 40].

2.6.1.1 Import danych oraz konfigurowanie lokalizacji elektrod

Surowy zapis cyfrowy importuje się do wybranego środowiska obliczeniowego, gdzie przypisuje się współrzędne przestrzenne do elektrod - bazując na opisanych wcześniej standardach, takich jak 10-20. Usuwa się także kanały nieużywane, puste, czy takie, które nie będą potrzebne do obliczeń i analizy [5, 40].

2.6.1.2 Filtrowanie cyfrowe

Przejście sygnału przez kombinację filtrów to podstawowy krok w kierunku uzyskania oczyszczonych z artefaktów i szumów danych.

1) Filtrowanie górnoprzepustowe (ang. high-pass filtering)
   Stosuje się go najczęściej, aby pozbyć się zakłócenia polegającego na przemieszczaniu się izolinii na wykresie w górę i w dół, zamiast utrzymywać ją w stałej pozycji poziomej

- spowodowane są one, np. przez powolne ruchy głowy, czy zmiany potliwości skóry [5, 9].

Działa poprzez przepuszczanie sygnałów o częstotliwościach wyższych od częstotliwości granicznej, jednocześnie tłumiąc składowe o częstotliwości niższej. Częstotliwość graniczna najczęściej ustawia się na poziomie od 0,1 Hz do 1 Hz [9].

Dla prostego analogowego filtra górnoprzepustowego rezystancyjno-pojemnościowego (ang. resistor-capacitor filter, RC) pierwszego rzędu częstotliwość graniczną można wyznaczyć ze wzoru [15]:

$$f_c = \frac{1}{2\pi RC},$$

gdzie:

f_c – częstotliwość graniczna,

R – rezystancja,

C – pojemność.

2) Filtrowanie dolnoprzepustowe (ang. low-pass filtering)

Analogicznie do filtrowania górnoprzepustowego - przepuszcza sygnały o częstotliwościach niższych od częstotliwości granicznej, a tłumi te o częstotliwościach wyższych. Filtr ten skutecznie radzi sobie z zakłóceniami o wysokiej amplitudzie, między innymi z zakłóceniami pochodzenia mięśniowego. Częstotliwość graniczną w tym przypadku oscyluje na przedziale od 30 Hz do 100 Hz [5].

W niniejszej pracy zastosowano implementację cyfrową filtrów pasmowoprzepustowych Butterwortha typu filtra o nieskończonej odpowiedzi impulsowej (ang. infinite impulse response, IIR). Dla każdego pasma EEG określono dolną i górną częstotliwość graniczną. Wzór ogólny transmitancji filtra cyfrowego zastosowany w implementacji ma postać:

$$H(z) = \frac{Y(z)}{X(z)} = \frac{\sum_{k=0}^{M} b_k z^{-k}}{\sum_{k=0}^{N} a_k z^{-k}},$$

gdzie:

$X(z)$ – transformata sygnału wejściowego,

$Y(z)$ – transformata sygnału wyjściowego,

b_k – współczynniki licznika transmitancji,

a_k – współczynniki mianownika transmitancji,

M, N – stopnie odpowiednio licznika i mianownika,

z^{-1} – opóźnienie sygnału o jedną próbkę.

Współczynniki a_k i b_k aplikacja wyznacza za pomocą funkcji *scipy.signal.butter* oddzielnie dla każdego pasma częstotliwości [16, 47]. Parametr `N` zastosowany w tym przypadku ma rząd równy 4, co dla filtra pasmowoprzepustowego prowadzi do uzyskania filtra ósmego rzędu [47].
Filtr stosowano dwukierunkowo za pomocą funkcji *scipy.signal.filtfilt* - najpierw zgodnie z kolejnością próbek, a następnie w kierunku przeciwnym, dzięki czemu przesunięcie fazowe wyniesie zero [48].

3) Filtrowanie zaporowe (ang. notch/band-stop filtering)
Filtr Notch służy najczęściej do eliminowania zakłóceń z sieci elektrycznej - zależnie od lokalnej sieci tłumi składową zakłóceń w pobliżu 50 lub 60 Hz [5].

2.6.1.3 Re-referencja

Proces polega na ponownym przeliczeniu potencjału elektrycznego elektrod względem nowego punktu odniesienia. Początkowo takim źródłem referencji mogła być pojedyncza elektroda, po zastosowaniu re-referencji wykorzystując jedną z metod - średnią wspólną (ang. common average reference, CAR) punktem odniesienia staje się średnia ze wszystkich elektrod [5].

2.6.1.4 Segmentacja sygnału

Segmentacja sygnału polega na podzieleniu ciągłego zapisu EEG na krótsze fragmenty czasowe, nazywane epokami (ang. epochs). Każda epoka powinna odpowiadać określonemu etapowi eksperymentu, bodźcowi lub reakcji badanego. Ułatwia to porównywanie fragmentów sygnału zarejestrowanych w podobnych warunkach oraz przygotowanie obserwacji wykorzystywanych w dalszej analizie [5, 27, 40].

Podział można wykonywać na podstawie znaczników zdarzeń zapisanych wraz z sygnałem EEG. Znacznik zawiera zazwyczaj kod zdarzenia oraz czas lub numer próbki, w której ono wystąpiło. Może oznaczać między innymi rozpoczęcie i zakończenie prezentacji bodźca, pojawienie się obrazu lub dźwięku, udzielenie odpowiedzi przez badanego albo przejście do kolejnej próby eksperymentalnej. Na podstawie tych znaczników z ciągłego zapisu wycina się fragmenty obejmujące określony przedział przed zdarzeniem i po nim [27, 40].

W badaniach nad emocjami początkiem epoki może być moment rozpoczęcia prezentacji materiału wywołującego określoną reakcję emocjonalną. Epoka obejmuje wówczas fragment sygnału zarejestrowany podczas oddziaływania bodźca, a przypisana do niej etykieta opisuje zakładaną kategorię emocji. Dłuższe próby mogą być następnie dzielone na krótsze okna czasowe wykorzystywane podczas ekstrakcji cech. Długość segmentów należy dostosować do celu analizy. Zbyt krótki fragment może nie zawierać wystarczającej ilości danych do wiarygodnego wyznaczenia właściwości częstotliwościowych, natomiast zbyt długi może obejmować zmiany stanu badanego, kilka różnych zdarzeń lub większą liczbę arte-

faktów [5, 27, 40]. Istotne jest również zachowanie informacji o uczestniku, numerze próby i rodzaju bodźca, aby możliwe było późniejsze prawidłowe rozdzielenie danych treningowych i testowych.

W zbiorze SEED-VII wykorzystanym w niniejszej pracy sygnał był już uporządkowany w postaci oddzielnych prób eksperymentalnych, odpowiadających prezentacji kolejnych materiałów wywołujących określone emocje [10]. Z tego powodu implementacja nie wymagała samodzielnego wycinania prób z ciągłego zapisu na podstawie znaczników zdarzeń. Strukturę wykorzystanych plików i sposób organizacji prób przedstawiono szerzej w podrozdziale 3.3.3.

Na potrzeby własnej ekstrakcji cech każdą próbę podzielono dodatkowo na niepokrywające się okna o długości czterech sekund. Niepełny fragment znajdujący się na końcu próby był pomijany. Dla każdego okna obliczano cechy w poszczególnych pasmach częstotliwości, a następnie wyniki agregowano do wektora o stałym wymiarze. Szczegółowe założenia segmentacji i filtracji opisano w podrozdziale 3.4.2, natomiast sposób agregacji cech w podrozdziale 3.4.5.

2.6.1.5 Korekcja linii bazowej

W analizach EEG związanych z prezentacją bodźca poszczególne segmenty sygnału mogą różnić się poziomem bazowym napięcia. Korekcja linii bazowej polega na obliczeniu średniego potencjału w wyznaczonym przedziale poprzedzającym stymulację i odjęciu tej wartości od całego przebiegu czasowego danego segmentu. Ustala się w ten sposób wspólny poziom odniesienia, co ułatwia porównywanie zmian potencjału między segmentami. W niniejszej aplikacji operacja ta nie jest wykorzystywana [27].

2.6.1.6 Identyfikowanie uszkodzonych kanałów i ich interpolacja

W wyniku różnych błędów losowych, elektrody w trakcie badania mogą źle zarejestrować sygnał. Takie kanały odrzuca się i omija w dalszej analizie lub stosuje się technikę, która rekonstruuje sygnał uszkodzonego kanału wykorzystując interpolację sferyczną, która obliczana jest na podstawie danych z elektrod działających poprawnie i sąsiadujących z uszkodzonym kanałem [5].

2.6.1.7 Odrzucanie próbek

Próbki, które są zbyt zanieczyszczone licznymi artefaktami odrzuca się, aby zwiększyć jakość analizy, bądź danych treningowych do modelu uczenia maszynowego. Takie próbki można odrzucić z pomocą specjalnych programów/bibliotek - ustalając konkretne progi błędu, bądź zrobić to ręcznie analizą wizualną [5, 17, 40].

2.6.1.8 Analiza składowych niezależnych (ang. independent component analysis, ICA)

Gdyby odrzucano wszystkie próbki, które posiadają artefakty powiązane z typowymi dla badań EEG elementami - takimi jak ruch gałek ocznych, czy skurcze mięśni, niewiele próbek

zostałoby gotowych do użytku. W tym celu stosuje się analizę składowych niezależnych, która wykorzystując statystykę i stosując obliczenia, wyodrębnia zmieszane sygnały, estymując składowe przy założeniu, że są statystycznie niezależne. Artefakty są usuwane, a sygnał zrekonstruowany przy użyciu projekcji wstecznej. Warto zaznaczyć, że bardzo ważne dla skutecznego procesu ICA jest wykonanie wcześniej opisanego filtrowania górnoprzepustowego, którego wykonanie według badań znacząco wpływa na otrzymanie czystszych składowych [9, 17].

2.6.2 Ekstrakcja cech

Po procesie preprocessingu sygnał wciąż będzie charakteryzował się niskim stosunkiem sygnału do szumu (ang. signal-to-noise ratio, SNR). Aby w pełni przygotować dane do pracy z modelami uczenia maszynowego wykorzystuje się proces ekstrakcji cech, który uwydatni najważniejsze cechy sygnału, które pomogą modelowi w rozpoznawaniu konkretnych emocji o wiele dokładniej. Wykorzystując matematyczne operacje i algorytmikę można to osiągnąć na kilka sposobów poprzez użycie metod ekstrakcji, redukując wymiarowość danych i wyodrębniając najważniejsze parametry badanej emocji [4, 5].

W pracy [10] przedstawiono trzy metody ekstrakcji cech, które warto przytoczyć:

1. Gęstość widmowa mocy (ang. power spectral density, PSD)
   Metoda PSD opisuje rozkład mocy sygnału względem częstotliwości. Dzięki niej wiemy jakie pasmo szczególnie wyróżnia się na tle innych. Do uzyskania wyników PSD używa się zazwyczaj kilku algorytmów bądź metod [5, 6]:
   - Periodogram wykorzystujący szybką transformatę Fouriera (ang. fast Fourier transform, FFT), obliczając kwadrat jej modułu i stosując odpowiednią normalizację [28].
   - Metodę Welcha również częściowo wykorzystującą Szybką Transformatę Fouriera, ale na sygnale, który wcześniej podzielono na nakładające się segmenty i nałożeniu okna czasowego. Po uzyskaniu periodogramów segmentów, uśrednia się wyniki [29].
   - Metodę Bartletta działającą na zasadzie metody Welcha z wykluczeniem nakładających się na siebie segmentów [29].

   PSD często wykorzystuje się przy badaniach zaburzeń uwagi, takich jak ADHD, czy ataków padaczkowych. Ze względu na swoją charakterystykę, której słabą stroną jest wyłapywanie mocy pasm na skrajnych jego stronach, nie jest najbardziej efektywną metodą przy badaniu emocji, choć wciąż szeroko wykorzystywana [5, 6].
2. Entropia różniczkowa (ang. differential entropy, DE)
   Entropia różniczkowa w odróżnieniu od gęstości widmowej mocy bardzo dobrze radzi sobie z cechami w niskich i wysokich częstotliwościach. W pracy [10] wskazano, że DE

pozwala równoważyć informacje o energii niskich i wysokich częstotliwości. Dzięki temu efektywniej różnicuje wzorce na przestrzeni tych pasm [4, 8].
Wzory dla entropii różniczkowej [8, 10]:

a) Klasyczny

$$h(X) = -\int_{-\infty}^{+\infty} f(x) \log\left(f(x)\right)\, dx,$$

gdzie:
$h(X)$ – entropia różniczkowa zmiennej losowej X,
$f(x)$ – funkcja gęstości prawdopodobieństwa,
x – wartość zmiennej losowej.

b) Uproszczony
Aby skorzystać ze wzoru uproszczonego, sygnał musi być stacjonarny i podlegać rozkładowi normalnemu. Tutaj pojawia się pewien paradoks, gdyż sygnał EEG jest skrajnie niestacjonarny. W tym przypadku stosuje się uproszczenie polegające na tym, że dzięki rozłożeniu sygnału na krótkie fragmenty, możemy go uznać za prawie stacjonarny [8, 10]:

$$h(X) = \frac{1}{2} \ln\left(2\pi e \sigma^2\right),$$

gdzie:
$h(X)$ – entropia różniczkowa sygnału,
σ^2 – wariancja sygnału,
e – liczba Eulera,
π – stała matematyczna.

3. Liniowy system dynamiczny (ang. linear dynamical system, LDS)
Choć niektórzy nie uznają LDS jako odrębną cechę, a raczej rozszerzenie, np. do DE i swego rodzaju filtrem, to wciąż w wielu źródłach możemy przeczytać o nim jako cecha. Z racji tego, że emocje człowieka są płynne i mogą zmieniać się w czasie - nie możemy uznawać nagłych skoków w wyekstrahowanych cechach jako odzwierciedlenie danej emocji [10].
Załóżmy, że po ekstrakcji dostajemy wektor z liczbami:

$$[3.2,\ 3.1,\ 3.5,\ \mathbf{9.0},\ 3.4]$$

Liczba o indeksie 3 znacząco różni się od pozostałych. Może to być zwykły artefakt, który błędnie będzie wskazywał na wzór emocji dla trenowanego modelu.
LDS stara się zapobiegać takim sytuacjom i analizując trendy wygładza sygnał w czasie [8, 10].

Korzystając z powyższych metod sygnał jest przygotowywany, aby ostatecznie mógł być w pełni zrozumiały i użyteczny dla modeli uczenia maszynowego. Z dwuwymiarowej macierzy z tysiącami próbek sygnału otrzymujemy zwięzły, zagregowany wektor cech.

2.7 Modele uczenia maszynowego

Praca wykorzystuje trzy modele uczenia nadzorowanego: SVM, Random Forest oraz XGBoost. Modele te uczą się zależności między cechami wejściowymi, a etykietami klas na podstawie oznaczonych danych treningowych. Wykorzystują wyuczone zależności do klasyfikacji nowych obserwacji.

SVM wyznacza granicę decyzyjną, dążąc do maksymalizacji marginesu rozdzielenia klas. Jej położenie zależy od wybranych obserwacji treningowych, nazywanych wektorami nośnymi. Zastosowanie funkcji jądra, np. radialnej funkcji bazowej (ang. radial basis function, RBF), umożliwia uzyskanie nieliniowych granic decyzyjnych. Ponieważ działanie SVM zależy od skali cech, przed treningiem zalecana jest ich standaryzacja [30].

Random Forest jest modelem zespołowym składającym się z wielu drzew decyzyjnych. Poszczególne drzewa uczone są na losowanych ze zwracaniem próbach danych, a podczas wyznaczania podziałów rozważane są losowe podzbiory cech. Zróżnicowanie drzew i połączenie ich wyników służą ograniczeniu wariancji predykcji. W zastosowanej implementacji scikit-learn uśrednia się prawdopodobieństwa klas przewidywane przez poszczególne drzewa [31].

XGBoost jest implementacją boostingu gradientowego. W wykorzystanym wariancie kolejne drzewa decyzyjne dodawane są sekwencyjnie, aby zmniejszać wartość funkcji straty zespołu. Funkcja celu uwzględnia również karę za złożoność drzew, co pomaga ograniczać przeuczenie. W odróżnieniu od lasu losowego kolejne drzewa zależą więc od wyników wcześniej zbudowanego zespołu [32].

2.8 Środowisko i narzędzia

Narzędzia, biblioteki i technologie wykorzystane w projekcie zestawiono w tabeli 2.4.

Tabela 2.4. Wykorzystane technologie, biblioteki i frameworki

Narzędzie / biblioteka	Wersja	Zastosowanie
Python	3.14.6	język programowania wykorzystany do implementacji aplikacji i modeli uczenia maszynowego
NumPy	2.4.4	operacje na tablicach i danych numerycznych
SciPy	1.18.0	odczyt plików `.mat` oraz filtracja sygnałów
pandas	3.0.5	zapis i przetwarzanie raportów zawierających metryki
scikit-learn	1.9.0	walidacja, skalowanie danych oraz implementacja modeli SVM i Random Forest
XGBoost	3.4.1	implementacja modelu boostingowego
joblib	1.5.3	serializacja i zapis modeli oraz innych obiektów
matplotlib	3.11.1	tworzenie wykresów oraz generowanie elementów raportów PDF
Streamlit	1.61.1	budowa interfejsu aplikacji webowej
pytest	9.1.1	automatyczne testowanie aplikacji

3. Metodyka i implementacja systemu

3.1 Wprowadzenie

Rozdział drugi skupiał się głównie na założeniach badawczych i podstawach EEG: mechanizmach powstania, rejestracji i przetwarzania sygnału. Niniejszy rozdział przedstawia praktyczną część pracy. W rozdziale przedstawiono zasoby, z których korzystano w czasie tworzenia projektu, zastosowaną metodykę i bezpośrednią implementację. Opis obejmuje również integrację systemu z aplikacją webową oraz konfigurację graficznego interfejsu użytkownika. Wiele używanych określeń potrzebnych do zrozumienia procesu zostało szczegółowo opisane w poprzednim rozdziale.

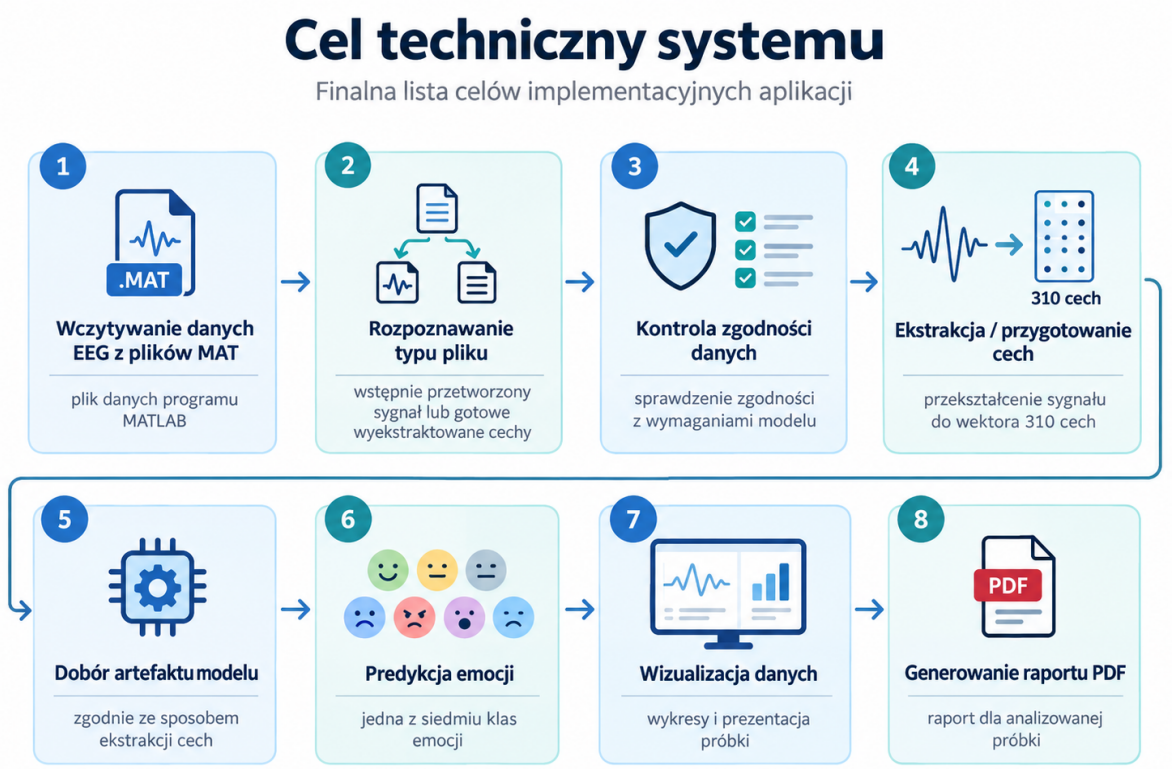
3.2 Cel projektu, wymagania oraz ograniczenia

3.2.1 Cel techniczny systemu

Finalna lista celów implementacyjnych obejmuje następujące funkcjonalności aplikacji:

- wczytuje dane EEG zapisane w plikach danych programu MATLAB,
- rozpoznaje typ pliku - wstępnie przetworzonego sygnału lub z gotowymi wyekstraktowanymi cechami,
- kontroluje, czy dane są zgodne z wymaganiami modelu,
- przekształca sygnał do wektora 310 cech,
- dobiera artefakt modelu zgodny ze sposobem ekstrakcji cech,
- zwraca predykcję jednej z siedmiu klas emocji,
- wizualizuje dane,
- pozwala pobrać raport w przenośnym formacie dokumentu PDF dla analizowanej próbki.

Kolejność realizacji wymienionych w powyższej liście punktowanej celów technicznych przedstawiono na rysunku 3.1.



Rysunek 3.1. Przepływ danych i główne funkcje aplikacji [43]

W związku z licznymi ograniczeniami projekt można uznać za prototyp badawczy, który jest gotowy na rozwój, ale nie należy go traktować jako narzędzia medycznego.

3.2.2 Wymagania funkcjonalne

Wymagania funkcjonalne aplikacji przedstawiono w tabeli 3.1.

3.2.3 Wymagania niefunkcjonalne

Aby zapewnić jakość i rzetelność działania aplikacji postawiono kilka wymagań, które zostały wdrożone do systemu.

Zaczynając od architektury - oddzielenie kodu modelu od interfejsu. Warstwa interfejsu powinna mieć rozdzieloną odpowiedzialność w systemie, dzięki czemu można ją w łatwy sposób modyfikować bez ingerowania w logikę i kod modelu.

Uczenie maszynowe powinno być zrealizowane w sposób umożliwiający łatwy ponowny trening modelu. Dzięki temu zachowana będzie elastyczność modyfikowania i ulepszania bądź eksperymentowania na tworzonym artefakcie, np. w przypadku rozszerzenia bazy danych treningowych.

Należy zwiększyć powtarzalność - determinizm obliczeń na każdym etapie przetwarzania - od wczytania danych, ekstrakcji cech, po określenie ziarna operacji pseudolosowych w czasie podziału danych, aby model uczył się i testował na dokładnie tych samych próbach.

Tabela 3.1. Wymagania funkcjonalne aplikacji

Funkcjonalność	Wymaganie	Realizacja
Funkcja 1	Wczytanie pliku .mat	Wybór pomiędzy wgraniem pliku lokalnie a przesłaniem przez przeglądarkę
Funkcja 2	Obsługa kilku prób w jednym pliku	Lista wykrytych prób i wybór w interfejsie
Funkcja 3	Obsługa formatu preprocessed i cech	Rozpoznanie formatu na podstawie kształtu tablicy i nazwy klucza
Funkcja 4	Wybór fragmentu sygnału	Narzędzie do wyboru zakresu mające wpływ jedynie na wykres sygnału *model korzysta z całej próby
Funkcja 5	Klasyfikacja emocji	model zgodny z rodzajem przygotowanych cech
Funkcja 6	Prezentacja danych	Wykres sygnału dla poszczególnych częstotliwości, mapa cieplna dla DE oraz rozkład predykcji dla emocji
Funkcja 7	Funkcja eksportu analizy	Generowanie raportu PDF
Funkcja 8	Obsługa błędów	Komunikaty informujące o niewłaściwych parametrach wprowadzanych danych

Aplikacja ma działać w architekturze lokalnej. Wynika to między innymi z badawczego charakteru całego projektu. Dane treningowe zostały udostępnione przez autorów jedynie w celach naukowych.

Interfejs użytkownika powinien w stosowny sposób informować o formacie przyjmowanych plików oraz wyświetlać komunikaty o błędach opartych na mechanizmie walidacji danych wejściowych.

3.2.4 Ograniczenia wejścia

Z powodu ograniczonego dostępu do danych treningowych, model oczekuje ich zgodnych ze zbiorem SEED-VII [10]. Wymagania danych na wejściu przedstawia tabela 3.2.

Tabela 3.2. Wymagania dotyczące danych wejściowych

Parametr	Wymaganie
Kanały	62 kanały w określonej kolejności
Częstotliwość próbkowania	200 Hz dla sygnału
Rodzaj danych	Wstępnie przetworzony sygnał lub wyekstraktowane cechy
Długość sygnału	Co najmniej 4 sekundy
Układ cech	[liczba_okien, 5, 62] lub [5, 62]

W przypadku, gdy plik zawiera metadane - częstotliwość próbkowania oraz nazwy kanałów mogą zostać zweryfikowane. Aplikacja pozwala na analizowanie nowych danych, jeśli zostały przygotowane zgodnie z wymaganiami.

3.3 Zbiór danych. Przygotowanie i charakterystyka

3.3.1 Źródło danych

W treningu i analizie wykorzystano zbiór SEED-VII [10].

Zbiór został przygotowany w warunkach laboratoryjnych przy wykorzystaniu sprzętu klasy medyczno-naukowej. Użyty standard układu elektrod to w tym przypadku 10-20. Istotne dla implementacji informacje przedstawia tabela 3.3.

Tabela 3.3. Charakterystyka zbioru SEED-VII [10]

Parametr	Wartość
Liczba badanych	20
Liczba prób na osobę	80
Sesje	4 sesje po 20 materiałów do stymulacji bodźcami
Klasy emocji	7 klas: ‘Happy’, ‘Sad’, ‘Fear’, ‘Disgust’, ‘Surprise’, ‘Anger’, ‘Neutral’
Liczba kanałów EEG	62
Częstotliwość próbkowania	200 Hz
Format plików	MATLAB (.mat)

Surowe sygnały rejestrowano z częstotliwością próbkowania 1000 Hz, lecz w implementacji niniejszej aplikacji korzystano z danych sygnałów wstępnie przetworzonych - dla których wartość częstotliwości zmniejszono do 200 Hz.

Wraz z opisanymi plikami dostarczono również dane dotyczące ruchu oczu, rejestrowane za pomocą okulografu Tobii Pro Fusion. Katalog EYE_raw obejmuje surowe dane okulograficzne, natomiast EYE_features zawiera wyodrębnione cechy związane między innymi ze średnicą źrenicy, fiksacjami, ruchami sakadowymi (niezwykle szybkie, skokowe przemieszczenia gałek ocznych realizowane według przygotowanego w mózgu planu motorycznego, które nakierowują obraz nowego obiektu na obszar siatkówki o najwyższej ostrości widzenia i mrugnięciami [53]). Dane te umożliwiają prowadzenie analizy multimodalnej, jednak nie były wykorzystywane w niniejszej pracy, ponieważ aplikacja została przeznaczona do analizy sygnałów EEG. Warto dodać, że pliki EYE mogą być wartościowym kierunkiem rozwoju aplikacji. SEED-VII został przygotowany jako zbiór multimodalny właśnie po to, aby informacje z EEG można było łączyć z cechami ruchu oczu. W pracy [10] wskazano, że obie modalności zawierają informacje wzajemnie się uzupełniające.

3.3.2 Próbki danych i etykiety

Jedną obserwacją modelu jest pełna próba eksperymentalna związana z prezentacją filmu. Własny ekstraktor wyznacza cechy w czterosekundowych oknach, które następnie są uśredniane w obrębie próby. Etykieta wynika z kategorii przypisanej materiałowi filmowemu w arkuszu kolejności bodźców. W eksperymencie nie wykorzystano ciągłych ocen intensywności emocji ani informacji o skuteczności wywołania emocji do zmiany etykiet. Klasyfikowany jest więc cel emocjonalny bodźca, który nie musi dokładnie odpowiadać stanowi uczestnika w całym czasie próby [10]. Liczbę prób przypadających na poszczególne klasy emocji zestawiono w tabeli 3.4.

Po agregacji charakterystyka prób przedstawia się następująco:

- 1600 prób (20 badanych poddanych 80 materiałom)
- 310 cech na obserwację
- macierz ‘X’ o kształcie [1600, 310]
- wektor etykiet ‘y’ o długości 1600,
- wektor ‘subjects’ identyfikujący badanego,
- wektor ‘trials’, który przechowuje numer materiału stymulującego

Tabela 3.4. Klasy emocji i liczba obserwacji [10]

Emocja	Indeks	Liczba prób na osobę	Suma obserwacji
Radość	0	12	240
Smutek	1	12	240
Strach	2	12	240
Wstręt	3	12	240
Zaskoczenie	4	12	240
Złość	5	12	240
Stan neutralny	6	8	160

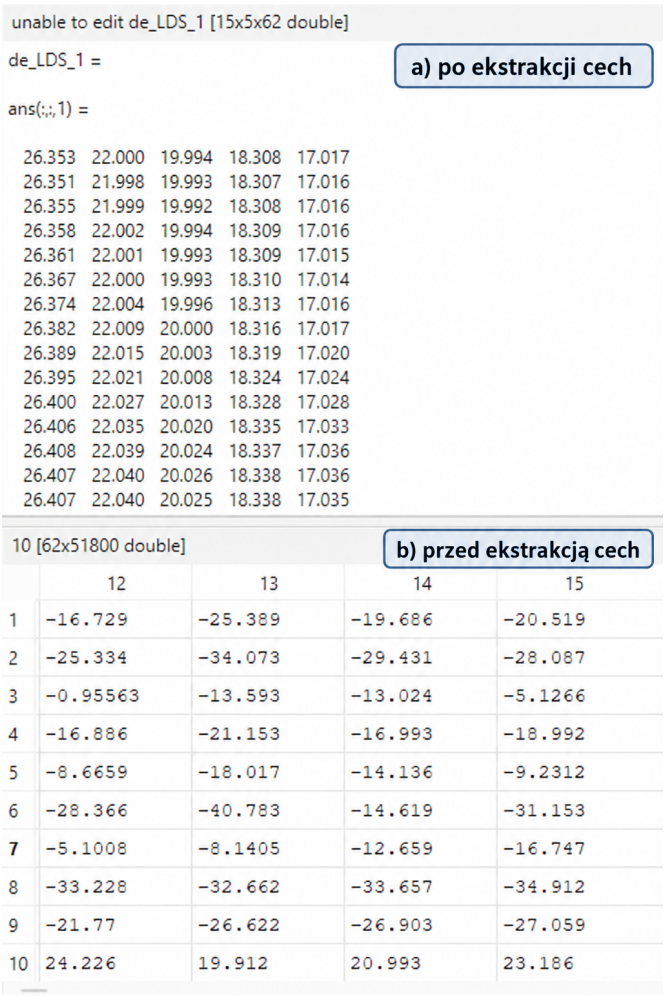
3.3.3 Struktura plików

Strukturę plików zawierających sygnał wstępnie przetworzony oraz gotowe cechy przedstawiono w tabeli 3.5.

W zależności od próby, liczby próbek i okien mogą się różnić od siebie. Z tego powodu przed uczeniem modelu wymagana jest agregacja do stałego wymiaru.

Tabela 3.5. Struktura plików danych [10]

Źródło	Przykładowy klucz	Kształt	Wyjaśnienie
EEG_preprocessed/ 1.mat	1	(62, 12200)	Sygnał wstępnie przetworzony, 62 kanały pierwszej próby
EEG_features/1.mat	de_LDS_1	(15, 5, 62)	Wyekstrahowane cechy - 15 okien, 5 pasm, 62 kanały



Rysunek 3.2. Przykładowa zawartość pliku przed i po ekstrakcji cech [10, 43]

3.3.4 Warianty zbioru treningowego

Projekt korzysta z dwóch wariantów zbioru treningowego:

1. Wariant referencyjny

   Użyte gotowe cechy de_LDS_* ze zbioru EEG_features, które są agregowane do wektora 310 cech, zapisany jako dataset.npz. Przykładową zawartość plików przed ekstrakcją cech (podpunkt b) i po ekstrakcji cech (podpunkt a) pokazano na rysunku 3.2.

2. Wariant własny

   Użyty sygnał z EEG_preprocessed, czyli sygnału wstępnie przetworzonego, który przechodzi przez kod z pliku feature_extraction (ekstrakcji cech), wynik jest agregowany i zapisywany jako dataset_own.npz. Wariant ten jest potrzebny, ponieważ w przypadku przyjmowania sygnału wstępnie przetworzonego zarówno aplikacja analizująca, jak i etap przygotowania danych muszą używać takiego samego algorytmu cech, aby nie otrzymać nieprawidłowych danych.

Listing 3.1. Mapowanie numeru próby na kategorię emocjonalną i etykietę klasy

```
def trial_to_emotion(trial_number: int) -> str:
    if not 1 <= trial_number <= N_TRIALS:
        raise ValueError("Numer próby poza zakresem")
    return TRIAL_EMOTIONS[trial_number - 1]

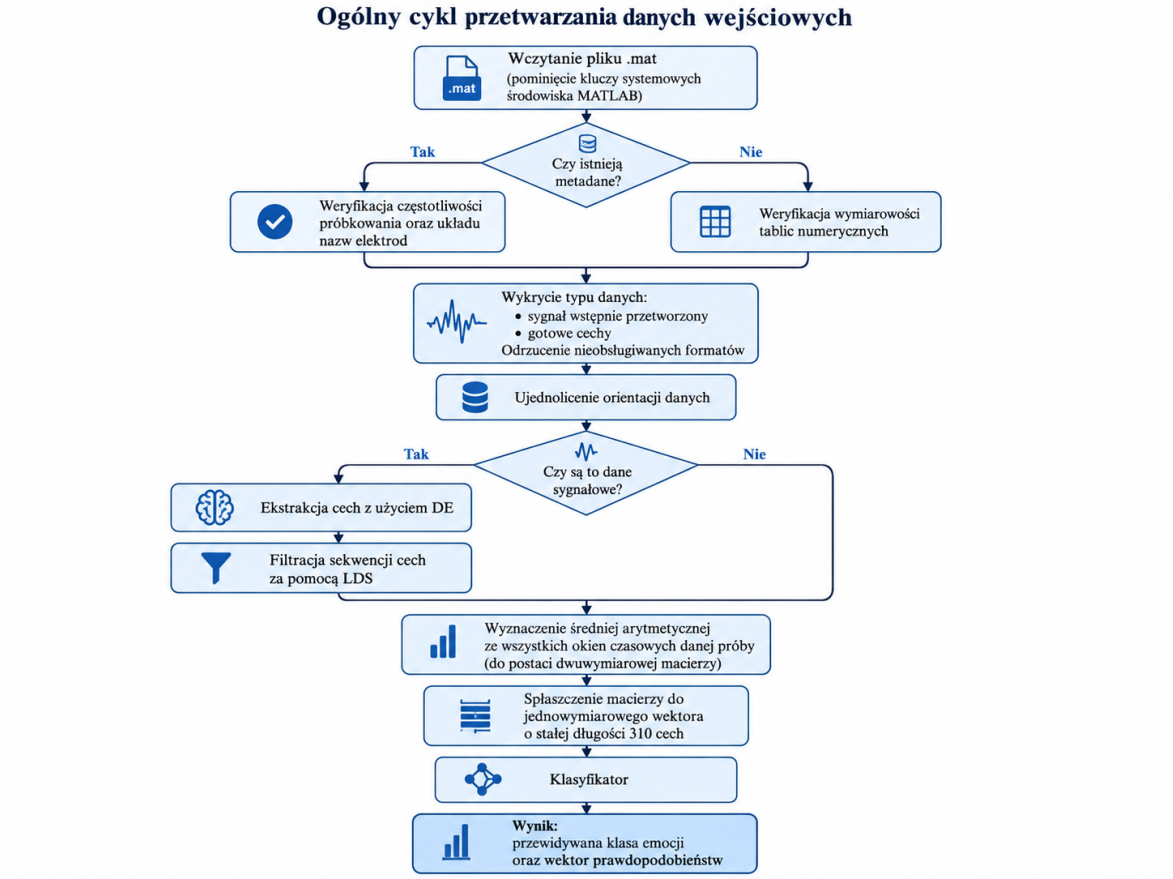
def trial_to_label(trial_number: int) -> int:
    return EMOTION_MAP[trial_to_emotion(trial_number)]
```

Etykiety wynikają z zawartej w plikach kolejności bodźców. Przypisywane są funkcjami pokazanymi w listingu 3.1 przed treningiem modelu.

3.4 Przetwarzanie i ekstrakcja cech

3.4.1 Ogólny przebieg przetwarzania

Ogólny cykl przetwarzania rozpoczyna się wczytaniem pliku o formacie .mat z pominięciem kluczy systemowych środowiska MATLAB. Następnie, jeśli istnieją metadane, weryfikowana jest zgodność częstotliwości próbkowania oraz układ nazw elektrod, w przeciwnym wypadku weryfikowana jest wymiarowość tablic numerycznych. Wykrywany jest typ danych - sygnał wstępnie przetworzony, bądź gotowe cechy oraz odrzucane są nieobsługiwane formaty. Ujednolicana jest orientacja danych, po czym w przypadku danych sygnałowych wykonuje się ekstrakcję cech (z użyciem DE) dla sygnału, a otrzymana sekwencja podlega filtracji za pomocą LDS. Następnym krokiem jest wyznaczenie średniej arytmetycznej ze wszystkich okien czasowych danej próby do postaci dwuwymiarowej macierzy, która zostaje spłaszczona do jednowymiarowego wektora o stałej długości 310 cech. Tak przygotowany wektor trafia na wejście klasyfikatora, który zwraca przewidywaną klasę emocji oraz wektor prawdopodobieństw. Przebieg ten obrazuje rysunek 3.3.



Rysunek 3.3. Ogólny cykl przetwarzania danych wejściowych [43]

3.4.2 Segmentacja i filtracja pasmowa

Zważając na to, że aplikacja oprócz gotowych plików cech przyjmuje również sygnał wstępnie przetworzony, podjęto kilka ważnych decyzji dotyczących sposobu segmentacji oraz filtracji:

- Długość pojedynczego okna ustalono na 4 sekundy - wybór był motywowany oryginalnym sposobem segmentacji zgodnym z bazą SEED-VII, tj. przykładowa próba trwająca 61 sekund została podzielona na 15 okien cech, gdyż 61//4 = 15 [10]
- Liczba próbek w oknie jest równa 800 dla przyjętej częstotliwości próbkowania sygnału 200 Hz co oznacza 800 punktów pomiarowych dla każdego z 62 kanałów.
- Zrezygnowano z nakładających się okien zgodnie z zasobami źródłowymi. Może to zapobiegać niepotrzebnemu, sztucznemu powiązaniu ze sobą sąsiednich wektorów cech [10].
- Niepełna końcówka próby jest pomijana. Jeżeli badana próba filmowa ma długość, która nie jest wielokrotnością 4 sekund - niepełny fragment na końcu sygnału jest odrzucany.
- Nim sygnał zostanie podzielony na próbki zostaje on przefiltrowany, aby uniknąć zniekształcania brzegów, gdyż filtry pasmowoprzepustowe powodują artefakty na początku i końcu filtrowanego fragmentu. Gdyby najpierw zostało wykonanie podzielenie mate-

riału na fragmenty, a dopiero potem filtracja, każda próbka posiadałaby zniekształcenia na brzegach [48].

- Zastosowanie filtracji zero-fazowej oraz filtru Butterwortha [47, 48] - zastosowanie filtru Butterwortha wyodrębni pasma częstotliwości bez generowania zakłóceń. Rząd czwarty jest dobrze wyśrodkowaną opcją - nie odcina fal zbyt łagodnie przepuszczając niepożądane elementy z innych fal, a jednocześnie nie tnie ich zbyt ostro, dzięki czemu jest realizowany szybko i stabilnie. Zakresy pasm zastosowane w implementacji zestawiono w tabeli 3.6; są one analogiczne do zakresów przedstawionych w części teoretycznej.

Tabela 3.6. Pasma częstotliwości [1, 10]

Pasmo	Zakres [Hz]
Delta	1-4
Theta	4-8
Alpha	8-14
Beta	14-31
Gamma	31-50

Filtrację pasmową oraz obliczanie DE w kolejnych oknach czasowych przedstawiono w listingu 3.2.

Listing 3.2. Filtracja pasmowa i obliczanie DE w oknach czasowych

```
window_len = WINDOW_SECONDS * fs
n_windows = signal.shape[1] // window_len
band_signals = [
    bandpass_filter(signal, low, high, fs)
    for low, high in FREQ_BANDS.values()
]

for window in range(n_windows):
    start = window * window_len
    stop = (window + 1) * window_len
    for band, band_signal in enumerate(band_signals):
        features[window, band, :] = differential_entropy(
            band_signal[:, start:stop]
        )
```

3.4.3 Entropia różniczkowa (DE)

DE oblicza się na podstawie wariancji próbek w oknie, obliczanej w każdym paśmie i kanale, przy założeniu rozkładu Gaussa [8, 10]. Wariancja ograniczona jest od dołu przez 1e-12. Sposób obliczania DE na podstawie wariancji sygnału przedstawiono w listingu 3.3.

Listing 3.3. Obliczanie DE na podstawie wariancji sygnału

```
def differential_entropy(signal: np.ndarray) -> np.ndarray:
    variance = np.var(signal, axis=-1)
    variance = np.maximum(variance, 1e-12)
    return 0.5 * np.log(2 * np.pi * np.e * variance)
```

3.4.4 Wygładzanie LDS

Po obliczeniu DE każda z 310 sekwencji cech zostaje niezależnie wygładzona. Wygładzanie ogranicza wpływ krótkotrwałych wahań wartości DE, które nie muszą odzwierciedlać rzeczywistych zmian stanu emocjonalnego [8, 10]. Operacja nie zmienia wymiarów danych - jej wynikiem pozostaje tablica o kształcie (liczba_okien, 5, 62). W implementacji zastosowano liniowy model przestrzeni stanów typu błądzenia losowego. Ukryty stan reprezentuje wygładzoną wartość cechy, natomiast wartość DE obliczona dla danego okna jest traktowana jako obserwacja obciążona szumem. Filtr Kalmana wyznacza estymaty w przejściu w przód [41], które są następnie korygowane w przejściu wstecznym za pomocą wygładzania Rauch-Tung-Striebela [42].

3.4.5 Agregacja do stałego wymiaru

Wykorzystywane klasyfikatory wymagają stałej liczby cech. Dla każdej próby uśredniono sekwencję cech wzdłuż osi okien czasowych, otrzymując macierz 5 x 62. Następnie spłaszczono ją do wektora 310 wartości, zachowując kolejność pasm i kanałów. Agregacja usuwa informacje o kolejności okien i zmianach cech w czasie. Implementację agregacji cech do wektora 310 wartości przedstawiono w listingu 3.4.

Listing 3.4. Agregacja cech próby do wektora 310 wartości

```
if trial_data.ndim == 2:
    trial_data = trial_data[np.newaxis, ...]
if trial_data.shape[1:] != (N_BANDS, N_CHANNELS):
    raise ValueError("Nieprawidłowy kształt cech")
return trial_data.mean(axis=0).flatten()
```

3.5 Modele uczenia maszynowego

Aby nie opierać ostatecznych wniosków tylko na jednym algorytmie, porównano trzy różne rodziny modeli. Zastosowane modele i ich parametry zestawiono w tabeli 3.7.

Tabela 3.7. Modele i parametry zastosowane w implementacji

Model	Główne parametry zastosowane w implementacji	Opis wyboru
SVM	RBF, “C=10”, “gamma=’scale’”	Odpowiedni dla danych wielowymiarowych
Random Forest	500 drzew, “random_state=42”	model zespołowy, nie wymaga skalowania, zdolny do modelowania zależności nieliniowych
XGBoost	300 estymatorów, “max_depth=6”, “learning_rate=0.1”, “multi:softprob”	wykorzystanie technologii boostingu, rozkład prawdopodobieństwa bezpośrednio dla siedmiu klas

Wynik uzyskany na próbach wykorzystanych do treningu nie pozwala wiarygodnie ocenić działania modelu na nowych danych. Dlatego modele oceniono za pomocą walidacji krzyżowej. Polega ona na podziale zbioru na części nazywane foldami. W kolejnych iteracjach model jest uczony od początku na części treningowej, a następnie sprawdzany na wydzielonej części testowej. W zastosowanych w tej pracy schematach każda próba trafia do części testowej dokładnie raz [44]. Sposób wydzielenia danych testowych ma znaczenie, ponieważ zbiór zawiera wiele prób tych samych uczestników oraz próby związane z tymi samymi bodźcami. Podział pojedynczych prób, oddzielenie wszystkich prób jednej osoby i oddzielenie prób przypisanych do tego samego bodźca odpowiadają na różne pytania o działanie modelu. Z tego powodu zastosowano trzy schematy walidacji, opisane w podrozdziale 3.5.1.

3.5.1 Walidacje modeli uczenia maszynowego

Schematy walidacji różnią się tym, które próby muszą pozostać oddzielone od danych treningowych. Poniżej omówiono sposób tworzenia podziałów oraz zakres oceny, jaki umożliwia każdy schemat.

1) **Walidacja zależna od uczestnika (ang. subject-dependent, SD)**
   W przyjętym wariancie próby wszystkich uczestników połączono w jeden zbiór i podzielono za pomocą klasy *StratifiedKFold*(). Zastosowano pięć foldów: w każdej iteracji 1280 prób służyło do treningu, a pozostałe 320 do testu. Podział jest stratyfikowany, czyli stara się zachować podobny udział poszczególnych klas emocji w każdym foldzie. Nie uwzględnia natomiast identyfikatorów uczestników ani bodźców. Oznacza to, że próby tych samych osób i próbki związane z tymi samymi bodźcami mogą występo-

wać zarówno w części treningowej, jak i testowej. Nie trenowano osobnego modelu dla każdej osoby [44].

2) **Walidacja z wyłączeniem jednego uczestnika (ang. leave-one-subject-out, LOSO)**
W tym schemacie zastosowano klasę *LeaveOneGroupOut*(). Grupą są wszystkie próby jednego uczestnika. W każdej iteracji 80 prób jednej osoby stanowi część testową, a 1520 prób pozostałych 19 osób, część treningową. Procedurę powtarza się 20 razy, tak aby każdy uczestnik dokładnie raz był osobą testową. Dzięki temu żadne próby testowanej osoby nie są używane do treningu w tej iteracji. Identyfikatory bodźców mogą jednak występować w obu częściach podziału [10, 44].

3) **Walidacja niezależna od bodźców (ang. stimulus-independent, SI)**
W tym schemacie zastosowano klasę *StratifiedGroupKFold*() z pięcioma foldami. Grupę wyznacza identyfikator bodźca, dlatego wszystkie próby związane z danym bodźcem trafiają do tej samej części podziału. Bodźce obecne w części testowej nie występują wówczas w części treningowej. Próby tych samych uczestników mogą natomiast znajdować się po obu stronach podziału. Metoda stara się zachować podobne proporcje klas w foldach, lecz ograniczenie dotyczące rozłączności bodźców nie zawsze pozwala uzyskać idealnie równy podział [44].

W implementacji liczba foldów w schematach SD i SI jest określona parametrem N_FOLDS=5. W LOSO liczba iteracji wynika z liczby uczestników i wynosi 20, dlatego parametr N_FOLDS nie jest w tym schemacie używany. W podziałach SD i SI ustawiono shuffle=True oraz stałe ziarno losowe 42, co umożliwia odtworzenie podziałów.

Dla każdego modelu predykcje wyznaczano funkcją *cross_val_predict*() z metodą predict_proba. Funkcja zwraca dla każdej próby prawdopodobieństwa klas uzyskane z modelu, który nie był trenowany na tej próbie. Są to predykcje spoza części treningowej (ang. out-of-fold, OOF). W LOSO do podziału przekazywano identyfikatory uczestników, a w SI - identyfikatory bodźców; wariant SD nie wymagał grupowania. Metryki obliczano z połączonych predykcji OOF wszystkich prób, a nie jako średnią metryk poszczególnych foldów. Taka ocena dotyczy jednak przyjętych schematów walidacji i nie zastępuje testu na niezależnym, zewnętrznym zbiorze danych [44].

Porównanie schematów walidacji przedstawia tabela 3.8.

Tabela 3.8. Porównanie sposobów walidacji

Walidacja	Klasa realizująca podział	Grupy	Liczba foldów
subject-dependent	StratifiedKFold	brak	5
LOSO	LeaveOneGroupOut	osoby	taka sama jak liczba osób badanych, czyli 20
stimulus-independent	StratifiedGroupKFold	bodźce	5

3.5.2 Metryki

Do oceny modeli wykorzystano zestaw uzupełniających się metryk obliczanych na podstawie predykcji uzyskanych poza częścią treningową. Sama trafność nie przedstawia wszystkich właściwości klasyfikatora, zwłaszcza gdy liczebność poszczególnych klas nie jest jednakowa. Z tego powodu uwzględniono również macro-F1, macierz pomyłek oraz wyniki dla poszczególnych uczestników lub bodźców [45]. Ocenę rozszerzono o przedziały ufności wyznaczane metodą bootstrapową oraz metryki wykorzystujące pełny rozkład prawdopodobieństw klas. Pozwalają one sprawdzić nie tylko poprawność końcowej klasyfikacji, lecz także stabilność wyników i zgodność deklarowanej przez model pewności z rzeczywistą trafnością [49, 50]. Zastosowane metryki przedstawiono poniżej:

1) Trafność (ang. accuracy)
   Odsetek poprawnie sklasyfikowanych prób [45]. Sposób jej obliczania przedstawiono w listingu 3.5.
2) Miara F1 uśredniona makro (ang. macro-averaged F1 score, macro-F1)
   Średnia wartości F1 obliczonych dla poszczególnych klas emocjonalnych [45], zobrazowana na listingu 3.5 wraz z accuracy.

   Listing 3.5. Obliczanie accuracy i macro-F1

```
predictions = classes[np.argmax(probabilities, axis=1)]

  accuracy = accuracy_score(y, predictions)

  f1_macro = f1_score(
      y,
      predictions,
      average="macro",
  )
```

   To właśnie average=”macro” powoduje policzenie F1 oddzielnie dla każdej klasy oraz ich nieważonej średniej [45]
3) Macierz pomyłek
   Obrazuje najczęściej mylone klasy emocji [45]. Jej wyznaczanie oraz zapis wykresu przedstawiono w listingu 3.6.

Listing 3.6. Wyznaczanie i zapis wykresu macierzy pomyłek

```
cm = confusion_matrix(
      y,
      predictions,
      labels=classes,
  )

  save_confusion_matrix(
      cm,
      f"{name} - {scheme} (accuracy={accuracy:.3f})",
      REPORTS_DIR / f"cm_{scheme}_{name}.png",
  )
```

Normalizację wierszy macierzy pomyłek przedstawiono w listingu 3.7.

Listing 3.7. Normalizacja wierszy macierzy pomyłek

```
cm_norm = cm.astype(float) / cm.sum(axis=1, keepdims=True)
```

Daje nam to wartości sumujące się do 1.

4) Accuracy mierzone osobno dla każdego badanego w LOSO
   Obliczanie i zapis accuracy dla poszczególnych osób lub bodźców przedstawiono w listingu 3.8.

Listing 3.8. Obliczanie i zapis accuracy dla poszczególnych osób lub bodźców

```
if scheme in {"loso", "stimulus_independent"}:
      grouping = subjects if scheme == "loso" else trials
      label = "subject" if scheme == "loso" else "trial"

      per_group = {
          int(group): accuracy_score(
              y[grouping == group],
              predictions[grouping == group],
          )
          for group in np.unique(grouping)
      }

      pd.Series(
          per_group,
          name="accuracy",
      ).rename_axis(label).to_csv(
          REPORTS_DIR / f"accuracy_by_{label}_{safe_name}.csv"
      )
```

Sposób tworzenia wykresu accuracy poszczególnych osób w LOSO przedstawiono w listingu 3.9.

Listing 3.9. Wizualizacja accuracy poszczególnych osób w LOSO

```
df = pd.DataFrame(frames)
  ax = df.plot.bar(figsize=(12, 5), width=0.8)

  ax.set_xlabel("Osoba")
  ax.set_ylabel("Accuracy")
  ax.set_title("LOSO - accuracy per osoba")
```

5) 95-procentowy przedział ufności trafności (ang. confidence interval, CI) wyznaczony metodą bootstrapową (ang. bootstrap) dla samych osób lub bodźców. Najpierw obliczane jest accuracy dla każdej osoby/bodźca, 10000 razy losując te wyniki ze zwracaniem. Kwantyle widoczne w kodzie wyznaczają 95 % przedziału [49]. Wyznaczanie przedziału ufności metodą bootstrapu grup przedstawiono w listingu 3.10.

Listing 3.10. Wyznaczanie przedziału ufności accuracy metodą bootstrapu grup

```
def clustered_accuracy_ci(y_true, y_pred, groups):
      scores = np.array([
          accuracy_score(
              y_true[groups == group],
              y_pred[groups == group],
          )
          for group in np.unique(groups)
      ])

      rng = np.random.default_rng(RANDOM_STATE)

      sampled = rng.choice(
          scores,
          size=(BOOTSTRAP_ITERATIONS, len(scores)),
          replace=True,
      )

      return tuple(
          float(value)
          for value in np.quantile(
              sampled.mean(axis=1),
              (0.025, 0.975),
          )
      )
```

6) Strata logarytmiczna (ang. log loss) jako diagnostyka rozkładu wyników "*predict_proba*" [45]. Sposób obliczania log-loss przedstawiono w listingu 3.11.

Listing 3.11. Obliczanie straty logarytmicznej (log-loss)

```
log_loss_value = log_loss(
      y,
      probabilities,
      labels=classes,
  )
"log_loss": round(
      float(log_loss(y, probabilities, labels=classes)),
      4,
  )
```

7) Wieloklasowy wynik Briera (ang. multiclass Brier score)
Określa jak bardzo przewidywany rozkład prawdopodobieństw różni się od prawdziwej klasy [45]. Obliczenia rozpoczynają się od zmiany prawdziwej klasy emocji na kodowanie „jeden z N” (ang. one-hot encoding), w którym jedna współrzędna ma wartość 1, a pozostałe 0. Kodowanie etykiet przedstawiono w listingu 3.12.

Listing 3.12. Kodowanie etykiet klas w reprezentacji one-hot

```
encoded = np.eye(len(classes))[y]
```

Następnie obliczany jest średni kwadratowy błąd całego wektora prawdopodobieństw, zgodnie z implementacją przedstawioną w listingu 3.13.

Listing 3.13. Obliczanie wieloklasowej miary Brier

```
brier_multiclass = np.mean(
      np.sum(
          (probabilities - encoded) ** 2,
          axis=1,
      )
  )
```

Brier score ocenia cały rozkład *predict_proba* (oszacowane prawdopodobieństwo, że próbka należy do poszczególnych klas) [45].

8) Oczekiwany błąd kalibracji (ang. Expected Calibration Error, ECE) obliczono zgodnie z implementacją przedstawioną w listingu 3.14 [50].

Listing 3.14. Obliczanie oczekiwanego błędu kalibracji (ECE)

```
def expected_calibration_error(
      y_true,
      y_pred,
      probabilities,
      bins=10,
  ):
      top_scores = probabilities.max(axis=1)
      correct = y_true == y_pred
      edges = np.linspace(0.0, 1.0, bins + 1)
      error = 0.0

      for lower, upper in zip(edges[:-1], edges[1:]):
          mask = (
              (top_scores > lower)
              & (top_scores <= upper)
          )

          if mask.any():
              error += (
                  mask.mean()
                  * abs(
                      correct[mask].mean()
                      - top_scores[mask].mean()
                  )
      return float(error)

  Wywołanie:

  ece = expected_calibration_error(
      y,
      predictions,
      probabilities,
  )
```

W tym przypadku ECE z 10 przedziałami porównuje średnią pewność najwyższej predykcji z rzeczywistą trafnością próbek w każdym przedziale [50].

3.5.3 Punkty odniesienia

Aby sprawdzić, czy użyte modele osiągają lepsze wyniki niż proste modele, bądź algorytmy użyto tzw. atrap (ang. Dummy), czyli właśnie modele bazowe oparte na prostych strategiach nie wykorzystując w żaden sposób sygnałów, bądź zależności między cechami EEG, a emocjami - są po prostu punktami odniesienia dla właściwych modeli [46].

W tym przypadku użyto:

1) Model bazowy MostFrequent, zdefiniowany za pomocą klasy DummyClassifier, przedstawiono w listingu 3.15.

Listing 3.15. Definicja modelu bazowego MostFrequent

```
DummyClassifier(strategy="most_frequent")
```

Przewiduje on klasę, która najczęściej występuje w zbiorze treningowym [46].

2) Losowy model bazowy Stratified przedstawiono w listingu 3.16.

Listing 3.16. Definicja losowego modelu bazowego Stratified

```
DummyClassifier(
      strategy="stratified",
      random_state=RANDOM_STATE,
  )
```

Model Stratified losuje przewidywaną klasę na podstawie proporcji klas ze zbioru treningowego [46].

3.5.4 Wybór emocji przez aplikację i walidację

Aby zachować pewność, że użytkownik widzi wyniki zgodne z działaniem systemu, klasa musi być wybierana w taki sam sposób zarówno w aplikacji, jak i podczas walidacji.

Takie działanie zapewnia wybór klasy z najwyższą wartością zwróconą przez *predic t_proba*.

Porównanie reguł w aplikacji i podczas walidacji:

1) Wariant stosowany podczas walidacji przedstawiono w listingu 3.17.

Listing 3.17. Wyznaczanie etykiet klas na podstawie predykcji OOF

```
probabilities = cross_val_predict(
      model,
      X,
      y,
      cv=cv,
      groups=cv_groups,
      method="predict_proba",
  )

  predictions = classes[np.argmax(probabilities, axis=1)]
```

Pozycja, która osiąga największy wynik jest znajdowana przez *np.argmax*(). Na tych etykietach oblicza się później accuracy, F1 i macierz pomyłek.

2) Wyznaczanie prawdopodobieństw klas w aplikacji przedstawiono w listingu 3.18.

Listing 3.18. Wyznaczanie prawdopodobieństw klas dla wektora cech wejściowych

```
probabilities = model.predict_proba(
      np.asarray(vector).reshape(1, -1)
  )[0]
```

Następnie interfejs wybiera klasę o najwyższym przewidywanym prawdopodobieństwie, co przedstawiono w listingu 3.19.

Listing 3.19. Wybór klasy o najwyższym przewidywanym prawdopodobieństwie

```
best_idx = int(np.argmax(proba))
  conf = float(proba[best_idx])
  pred_eng = EMOTION_NAMES[int(classes[best_idx])]
```

Jak można zauważyć w obu przypadkach wybierana jest klasa emocji o najwyższej wartości *predict_proba*.

Warto zaznaczyć, że procent jaki pokazuje się na wyjściu nie zawsze odpowiada rzeczywistej częstości poprawnych klasyfikacji. Samo *predict_proba* oznacza to, jak model rozdzielił swoje wyniki między klasy, następnie accuracy - jak często końcowa odpowiedź była poprawna, a ECE i Brier score sprawdzają, czy procentowe wartości odpowiadają rzeczywistej częstości poprawnych odpowiedzi. Aby interpretować wyniki dosłownie, model musiałby być bardzo dobrze skalibrowany [45, 50].

3.5.5 Wybór i zapis modelu

Model wdrożeniowy (czyli finalny model używany przez aplikację do klasyfikowania nowych sygnałów EEG) jest wybierany na podstawie F1-macro w schemacie LOSO, po wyborze jest on ponownie trenowany na wszystkich obserwacjach i wraz z nim zapisywane są także: wersja artefaktu, nazwa oraz parametry modelu, mapa emocji, rodzaj cech, schemat i wyniki wyboru, metryki, wersje bibliotek oraz odcisk danych treningowych (ang. training-data fingerprint). Aby modelu nie trzeba było trenować po każdym uruchomieniu aplikacji, jest on serializowany przez bibliotekę joblib w formie artefaktu, czyli słownika zawierającego model oraz informacje o nim. Strukturę artefaktu w formie słownika przedstawiono w listingu 3.20.

Listing 3.20. Struktura artefaktu modelu i jego metadanych

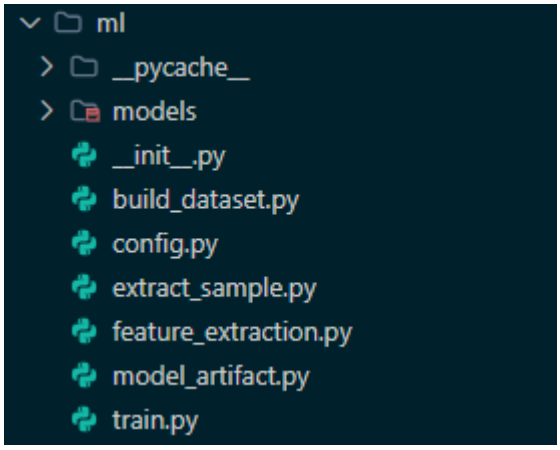
```
artifact = {
      "model": best_model,
      "model_name": best_name,
      "selection_scheme": criterion,
      "selection_metric": selection_metric,
      "emotion_map": EMOTION_MAP,
      "feature_type": feature_type,
      "data_summary": {...},
      "library_versions": library_versions(),}
```

Aplikacja weryfikuje metadane opisujące zapisany model i sprawdza, czy pasuje do aktualnej wersji aplikacji.

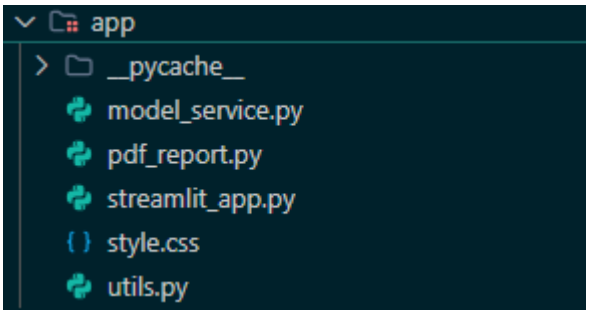
3.6 Architektura i implementacja aplikacji webowej

Architekturę aplikacji podzielono na dwie główne części: warstwę odpowiedzialną za przygotowanie danych i modele uczenia maszynowego oraz warstwę obsługującą działanie apli-

kacji webowej. Rozdzielenie logiki analitycznej od interfejsu użytkownika ułatwia rozwijanie, testowanie i utrzymywanie poszczególnych elementów systemu.



Rysunek 3.4. Struktura pakietu ml



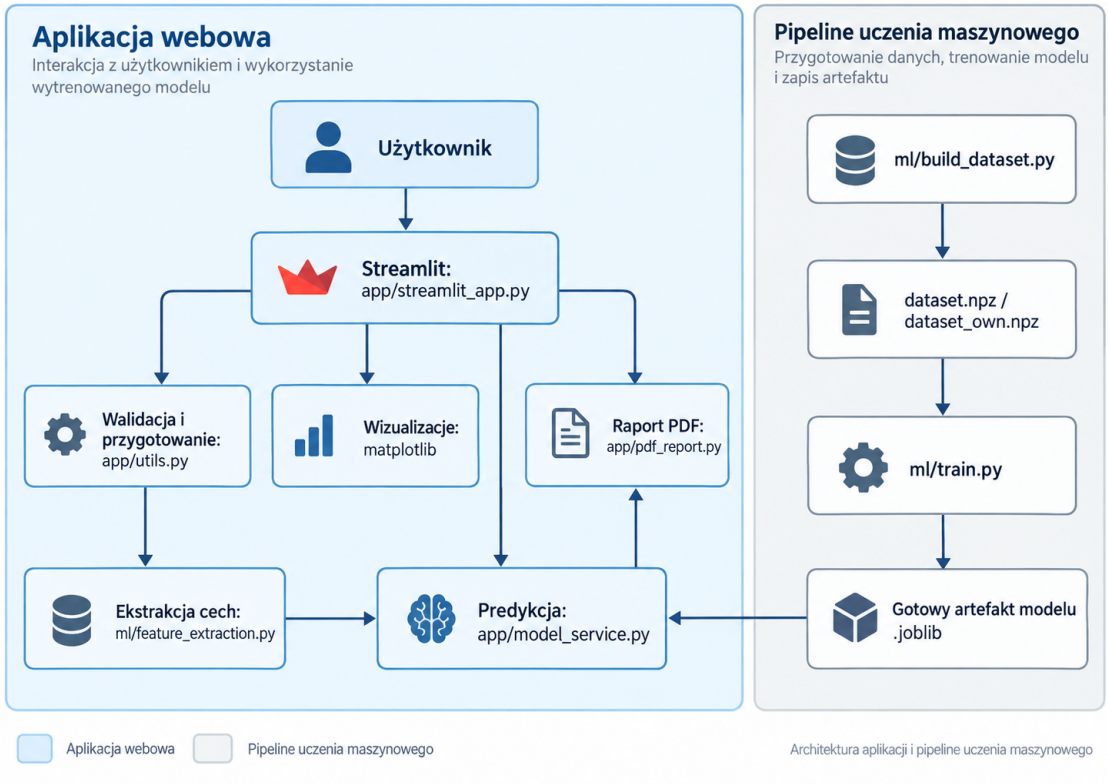
Rysunek 3.5. Struktura pakietu app

Pakiet ml obejmuje elementy odpowiedzialne za przygotowanie danych, ekstrakcję cech, trenowanie i walidację modeli oraz zapis ich artefaktów; jego strukturę przedstawiono na rysunku 3.4. Pakiet app odpowiada natomiast za wczytywanie i walidację plików użytkownika, dobór modelu, wykonanie predykcji, obsługę interfejsu oraz generowanie raportów; jego strukturę pokazano na rysunku 3.5. Szczegółowy podział odpowiedzialności poszczególnych modułów obu pakietów zestawiono w tabeli 3.9.

Rysunek 3.6 przedstawia ścieżkę offline treningu oraz online predykcji.

Tabela 3.9. Moduły aplikacji i ich odpowiedzialności

Moduł	Odpowiedzialność
ml/config.py	zawiera stałe, ścieżki, pasma, kanały i mapowanie emocji
ml/feature_extraction.py	ekstrakcja cech (filtracja pasmowa, podział na okna czasowe, DE, LDS)
ml/build_dataset.py	budowa macierzy uczących i agregacja prób
ml/train.py	trening modeli, walidacja, raporty metryk oraz zapis artefaktów
ml/model_artifact.py	tworzenie metadanych - wersja artefaktu, fingerprint zbioru, wersje bibliotek
app/utils.py	odczyt danych wejściowych .mat, walidacja, wykrywanie rodzaju pliku (sygnał/cechy), przygotowanie wektora cech i przekazanie go do modelu
app/model_service.py	kontrola zgodności artefaktu i wykonanie predykcji (*predict_proba*)
app/streamlit_app.py	sterowanie interfejsem, pipeline
app/pdf_report.py	tworzenie raportu PDF
ml/extract_sample.py	pomocniczy interfejs wiersza poleceń (ang. command-line interface, CLI) do wycinania danych .mat, niezintegrowany z interfejsem aplikacji



Rysunek 3.6. Przepływ przygotowania modelu i analizy pliku wejściowego [43]

3.6.1 Wczytywanie i walidacja pliku

Aplikacja przyjmuje na wejście pliki o rozszerzeniu .mat. Są one odczytywane za pomocą funkcji *scipy.io.loadmat* zwracającej słownik, w którym nazwy zmiennych zapisanych w pliku stanowią klucze, a odpowiadające im tablice są reprezentowane jako obiekty NumPy. Pomijane są klucze techniczne generowane przez MATLAB i SciPy - ich nazwy zaczynają się od __, np. __header__ lub __globals__. Wczytywanie tablic danych i pomijanie tych kluczy przedstawiono w listingu 3.21.

Listing 3.21. Wczytywanie tablic danych z pliku MAT

```
data = scipy.io.loadmat(file)

  arrays = {
      key: value
      for key, value in data.items()
      if not key.startswith("__")
  }
```

Aplikacja potrafi rozpoznawać nazwy metadanych w kilku wariantach - częstotliwość próbkowania pod kluczem fs, srate, sampling_rate, sr oraz nazw kanałów pod kluczami channels, clab, channel_names, bądź chan_names. Metadane są sprawdzane pod warunkiem, że istnieją w pliku. Aplikacja sprawdza także, czy częstotliwość wynosi 200 Hz i czy podano 62 kanały w kolejności zgodnej z SEED-VII. Jeżeli metadanych brakuje w pliku, aplikacja rozpoznaje format na podstawie kształtu i wartości tablicy - nie jest jednak w stanie potwierdzić częstotliwości próbkowania oraz kolejności elektrod.

Danymi wejściowymi mogą być zarówno wstępnie przetworzony sygnał EEG, jak i gotowe cechy. Sposób rozpoznawania ich formatu i walidacji przedstawiono w listingu 3.22.

Listing 3.22. Rozpoznawanie formatu danych i walidacja sygnału lub cech

```
if array.ndim == 3 and array.shape[1:] in (
    (N_BANDS, N_CHANNELS),
    (N_CHANNELS, N_BANDS),
):
    validate_feature_values(array, key=key)
    return "features"

if array.ndim == 2 and array.shape[0] == N_CHANNELS:
    validate_signal_properties(array, key=key)
    return "signal"
```

Tablice, które zawierają wartości nieliczbowe (ang. not a number, NaN), dodatnią lub ujemną nieskończoność, a także te, których nazwa wskazuje na cechy PSD (model został wytrenowany na cechach DE, wygładzanych LDS) są odrzucane. Niedopuszczalne jest także przyjęcie danych okulograficznych, które rozpoznawane są ze względu na obecność wymiaru zawierającego 33 parametry.

Aplikacja oblicza także odchylenie standardowe, na podstawie którego odrzuca sygnały, które mogły zostać zapisane w woltach, zamiast mikrowoltów lub takie, które wskazują na silny szum/artefakty.

Jeżeli wgrana próba zawiera zarówno cechy w wariancie de_LDS_*, jak i de_*, wybierany jest wariant LDS w celu uniknięcia ich powielenia.

Należy zaznaczyć iż przyjęcie pliku nie oznacza jednocześnie, że jest fizjologicznie prawidłowy - podane wymagania są jedynie kontrolą technicznej spójności.

3.6.2 Dobór właściwego modelu

Projekt wykorzystuje dwie ścieżki przetwarzania danych. Pierwsza ścieżka wykorzystuje gotowe cechy DE z wygładzaniem LDS. Cechy każdej próby są uśredniane po oknach czasowych i spłaszczane do wektora 310 wartości, na podstawie których budowany jest dataset.npz, a model końcowy zapisywany jako artefakt best_model.joblib. Druga ścieżka wykorzystuje natomiast wstępnie przetworzony sygnał (EEG_preprocessed). Ekstraktor najpierw filtruje cały sygnał danej próby do pięciu pasm częstotliwości, a następnie dzieli przefiltrowane przebiegi na niepokrywające się 4-sekundowe okna, oblicza DE i stosuje LDS. Po uśrednieniu okien powstaje dataset_own.npz, a powstały artefakt zapisany jako best_model_own_features.joblib. Oba modele wykorzystują 310 cech DE wygładzanych metodą LDS, lecz nie są traktowane zamiennie - cechy referencyjne (de_LDS) i te obliczone przez własny ekstraktor mogą się od siebie różnić.

Artefakt modelu jest wybierany na podstawie formatu wejścia, zgodnie z regułą przedstawioną w listingu 3.23.

Listing 3.23. Dobór artefaktu modelu do formatu danych wejściowych

```
model_path, feature_type = (
      (OWN_MODEL_PATH, OWN_FEATURE_TYPE)
      if fmt == "signal"
      else (BEST_MODEL_PATH, FEATURE_TYPE)
  )
```

Jeśli artefakt wymagany dla danego wejścia nie istnieje, zgłaszany jest komunikat obsługiwany w sposób przedstawiony w listingu 3.24.

Listing 3.24. Obsługa braku artefaktu modelu

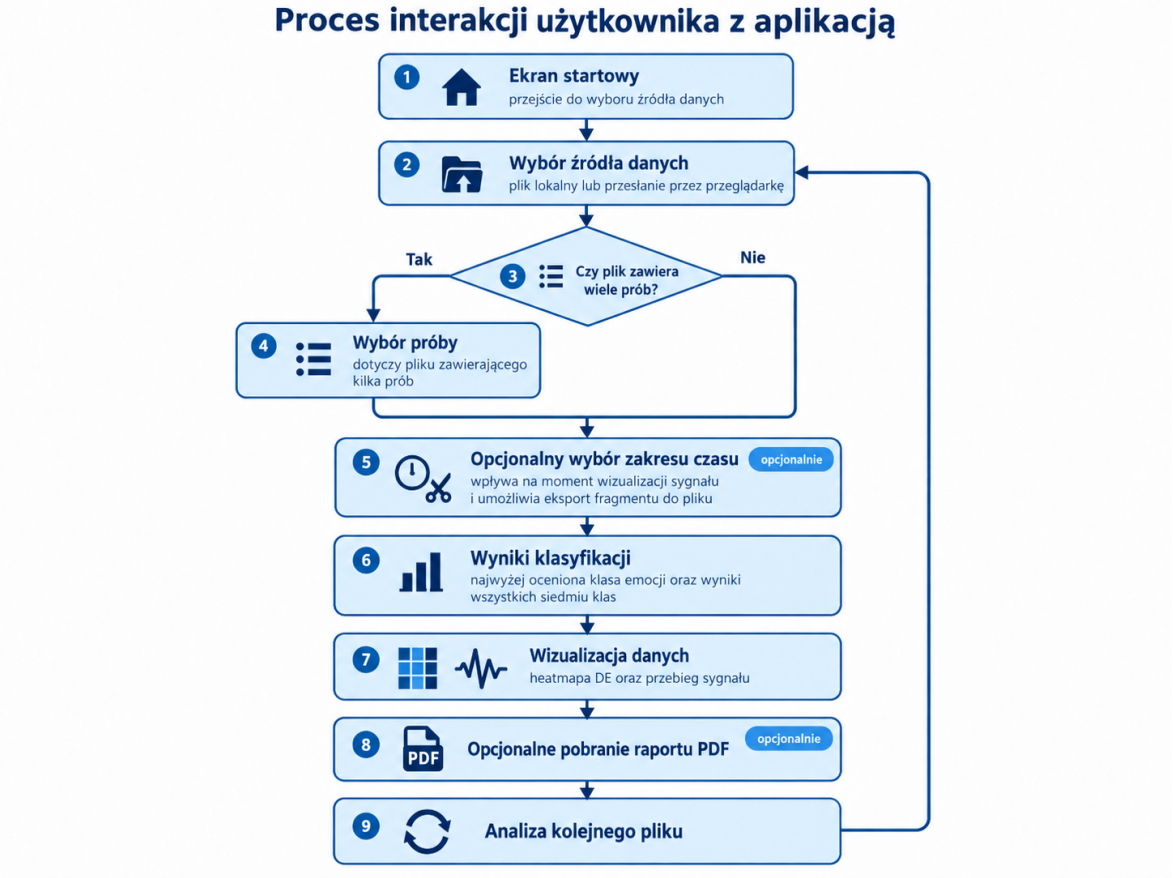
```
if artifact is None:
      raise MatFileError(
          f"Nie znaleziono modelu wymaganego dla tego wejścia:
{model_path}"
      )
```

Przed predykcją funkcja *validate_artifact*() sprawdza wersję struktury artefaktu, zgodność feature_type, mapy kodów emocji (przypisane indeksy), wybór modelu na podstawie LOSO, zestaw klas 0-6 oraz wymiar wejściowy równy 310 cech.

Przebieg i walidacje zapobiegają pozornie poprawnemu wynikowi spowodowanemu wyborem niewłaściwego modelu.

3.6.3 Przebieg interakcji użytkownika

Interakcja użytkownika z aplikacją została zaprojektowana jako spójny i intuicyjny proces, którego kolejne etapy przedstawiono na rysunku 3.7.



Rysunek 3.7. Proces interakcji użytkownika z aplikacją [43]

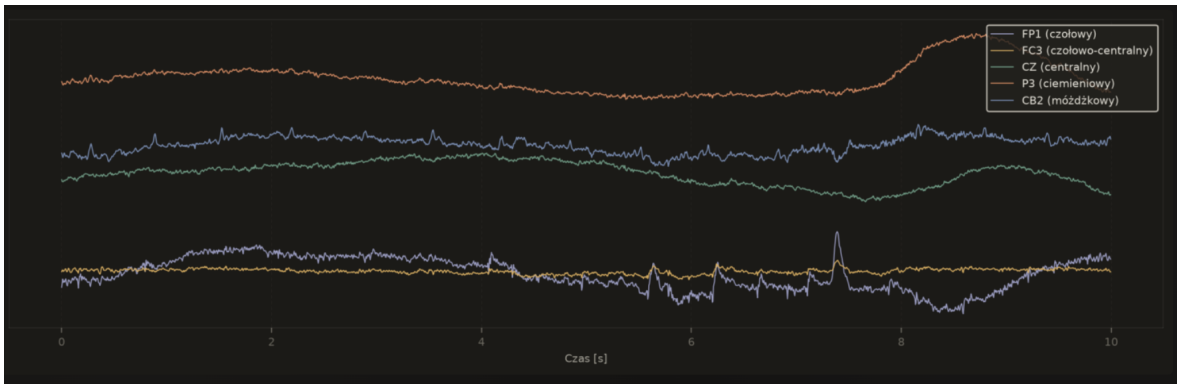
Użytkownik na początku zostaje przekierowany na ekran startowy, widoczny na rysunku 3.8. Po poprawnym przejściu przez proces importu plików uzyskuje planszę z klasyfikacją emocji przedstawioną na rysunku 3.9, heatmapę DE, możliwość eksportu fragmentu próbki oraz raportu PDF, a także w przypadku analizy pliku z sygnałem wstępnie przetworzonym – wizualizację sygnału wybranej próby przedstawioną na rysunku 3.10.



Rysunek 3.8. Interfejs aplikacji - strona startowa



Rysunek 3.9. Interfejs aplikacji - klasyfikacje emocji



Rysunek 3.10. Interfejs aplikacji - wybrany sygnał próby

Na rysunkach prezentujących interfejs można zauważyć wykorzystanie frameworku - Streamlit, który umożliwia użycie dwóch przydatnych mechanizmów - st.session_state oraz *st.cache_resource*. Zapobiegają one niepotrzebnej utracie danych lub kosztownemu tworzeniu obiektów.

st.session_state przechowuje stan interfejsu dla konkretnej sesji użytkownika między kolejnymi wykonaniami skryptu.

Inicjalizację stanu sesji aplikacji przedstawiono w listingu 3.25.

Listing 3.25. Inicjalizacja stanu sesji aplikacji

```
st.session_state.setdefault("data_loaded", False)
st.session_state.setdefault("uploader_key", 0)
```

Zapamiętywanie danych wczytanego pliku w stanie sesji przedstawia listing 3.26.

Listing 3.26. Zapamiętywanie danych wczytanego pliku w stanie sesji

```
  st.session_state.loaded_arrays = arrays
  st.session_state.source_name = uploaded.name
  st.session_state.data_loaded = True
```

W stanie sesji przechowuje się znacznik wczytania danych, tablice z pliku, nazwę źródła i identyfikator widżetu przesyłania.

Podczas resetu dane wejściowe są usuwane, a interfejs odświeżany w sposób przedstawiony w listingu 3.27.

Listing 3.27. Resetowanie danych wejściowych i odświeżanie interfejsu

```
  st.session_state.data_loaded = False
  st.session_state.loaded_arrays = None
  st.session_state.source_name = ""
  st.session_state.uploader_key += 1
  st.rerun()
```

Mechanizm *st.cache_resource* służy do przechowywania kosztownych zasobów, które mogą być wielokrotnie używane. W niniejszej aplikacji przykładem takiego zasobu jest artefakt modelu. Sposób jego wczytywania i buforowania pokazuje listing 3.28.

Listing 3.28. Wczytywanie i buforowanie artefaktu modelu

```
@st.cache_resource
  def load_artifact(model_path: Path):
      if not model_path.exists():
          return None
      return joblib.load(model_path)
```

4. Wyniki i analiza

Analiza i testy zostały przeprowadzone dla dwóch reprezentacji cech EEG, trzech klasyfikatorów, a także dwóch modeli bazowych przy wykorzystaniu trzech rodzajów walidacji. Każda reprezentacja obejmuje 1600 prób, które pochodzą od 20 uczestników badania, opisanych za pomocą 310 cech i przypisanych do jednej z siedmiu klas. Pojedyncza predykcja dotyczy całej próby. Wyniki w metrykach accuracy, macro-F1 oraz przedziałów accuracy zostały sprawdzone względem predykcji uzyskanych poza częścią treningową (ang. out-of-fold, OOF) [44].

OOF to predykcja, która została uzyskana poza częścią treningową dla danego podziału walidacji. Jest to sposób uzyskania uczciwszych predykcji walidacyjnych. Nie zastępuje jednak analizy na całkowicie niezależnym i zewnętrznym zbiorze danych. Znaczenie OOF jest zależne od walidacji, zgodnie z wcześniejszymi objaśnieniami [44]. Dla przykładu - OOF w walidacji LOSO: Dane jednego uczestnika w pierwszym foldzie stanowiły część testową, natomiast pozostałych 19 uczestników wykorzystano do treningu modelu. Następnie do części testowej wykorzystywana jest druga osoba, a pozostałe to dane treningowe. Proces powtarza się aż do dwudziestej osoby - tak, aby każdy uczestnik dokładnie raz stanowił osobę testową. Po zakończeniu procesu, predykcje testowe wszystkich osób są łączone. Otrzymujemy 1600 predykcji OOF - po jednej dla każdej próby. W każdym podziale model jest trenowany od początku. Oznacza to, że dla przykładowego accuracy na poziomie 24,25 %, spośród wszystkich 1600 połączonych predykcji LOSO, model poprawnie sklasyfikował dokładnie 388 prób.

4.1 Przygotowanie analizy

Tabela 4.1 zestawia trzy sposoby walidacji opisane w rozdziale trzecim oraz sposób podziału danych w każdym z nich. Walidacja subject-dependent dopuszczała obecność tych samych osób po obu stronach podziału (trening i test). LOSO pozwalała ocenić jak skuteczny jest model dla nowego uczestnika - którego dane nie występowały w zbiorze treningowym. Z kolei stimulus-independent ograniczał współdzielenie bodźców między częścią treningową i testową [44].

Tabela 4.1. Podział danych w poszczególnych walidacjach

Nazwa walidacji	Liczba podziałów (foldów)	Część treningowa/testowa	Co jest rozłączne?	Co może występować w obu częściach?
subject-dependent	5	1280 / 320 prób w podziale	Obserwacje	Osoby i identyfikatory bodźców
LOSO	20	1520 / 80 prób w podziale	Osoby	Identyfikatory bodźców
stimulus-independent	5	Zależnie od przydziału grup bodźców	Identyfikatory bodźców	Osoby

4.2 Zestawienie i porównanie klasyfikatorów oraz modeli bazowych

Wyniki dla wariantu referencyjnego przedstawiono w tabeli 4.2, natomiast wyniki dla wariantu własnego w tabeli 4.3. Pogrubieniem oznaczono najlepsze wyniki accuracy i macro-F1 wśród klasyfikatorów SVM, Random Forest i XGBoost dla każdego schematu walidacji.

Tabela 4.2. Wyniki klasyfikacji - wariant referencyjny

Nazwa walidacji	Klasyfikator	Accuracy [%]	95 % CI accuracy [%]	Macro-F1
SD	**SVM**	**30,31**	26,37–34,25	**0,2942**
SD	Random Forest	26,31	22,88–29,94	0,2560
SD	XGBoost	27,87	24,94–30,94	0,2794
SD	MostFrequent	15,00	15,00–15,00	0,0373
SD	Stratified	15,12	12,87–17,50	0,1476
LOSO	SVM	22,06	19,62–24,56	0,2083
LOSO	Random Forest	21,94	18,94–24,94	0,2021
LOSO	**XGBoost**	**22,25**	19,81–24,87	**0,2153**
LOSO	MostFrequent	15,00	15,00–15,00	0,0373
LOSO	Stratified	13,75	13,75–13,75	0,1347
SI	SVM	27,69	23,81–31,69	0,2690
SI	Random Forest	26,69	22,94–30,69	0,2583
SI	**XGBoost**	**28,88**	25,62–32,19	**0,2864**
SI	MostFrequent	12,50	6,25–20,00	0,0545
SI	Stratified	15,44	13,56–17,38	0,1502

Tabela 4.3. Wyniki klasyfikacji - wariant własny

Nazwa walidacji	Klasyfikator	Accuracy [%]	95 % CI accuracy [%]	Macro-F1
SD	**SVM**	**42,06**	38,44–45,38	**0,4097**
SD	Random Forest	32,19	28,81–35,25	0,3182
SD	XGBoost	36,75	32,31–40,88	0,3677
SD	MostFrequent	15,00	15,00–15,00	0,0373
SD	Stratified	15,12	12,87–17,50	0,1476
LOSO	SVM	23,69	21,25–26,25	0,2301
LOSO	Random Forest	22,63	20,19–25,25	0,2178
LOSO	**XGBoost**	**24,25**	21,94–26,94	**0,2371**
LOSO	MostFrequent	15,00	15,00–15,00	0,0373
LOSO	Stratified	13,75	13,75–13,75	0,1347
SI	**SVM**	**39,62**	33,87–45,50	**0,3797**
SI	Random Forest	33,31	29,00–37,94	0,3276
SI	XGBoost	37,81	33,75–42,06	0,3786
SI	MostFrequent	12,50	6,25–20,00	0,0545
SI	Stratified	15,44	13,56–17,38	0,1502

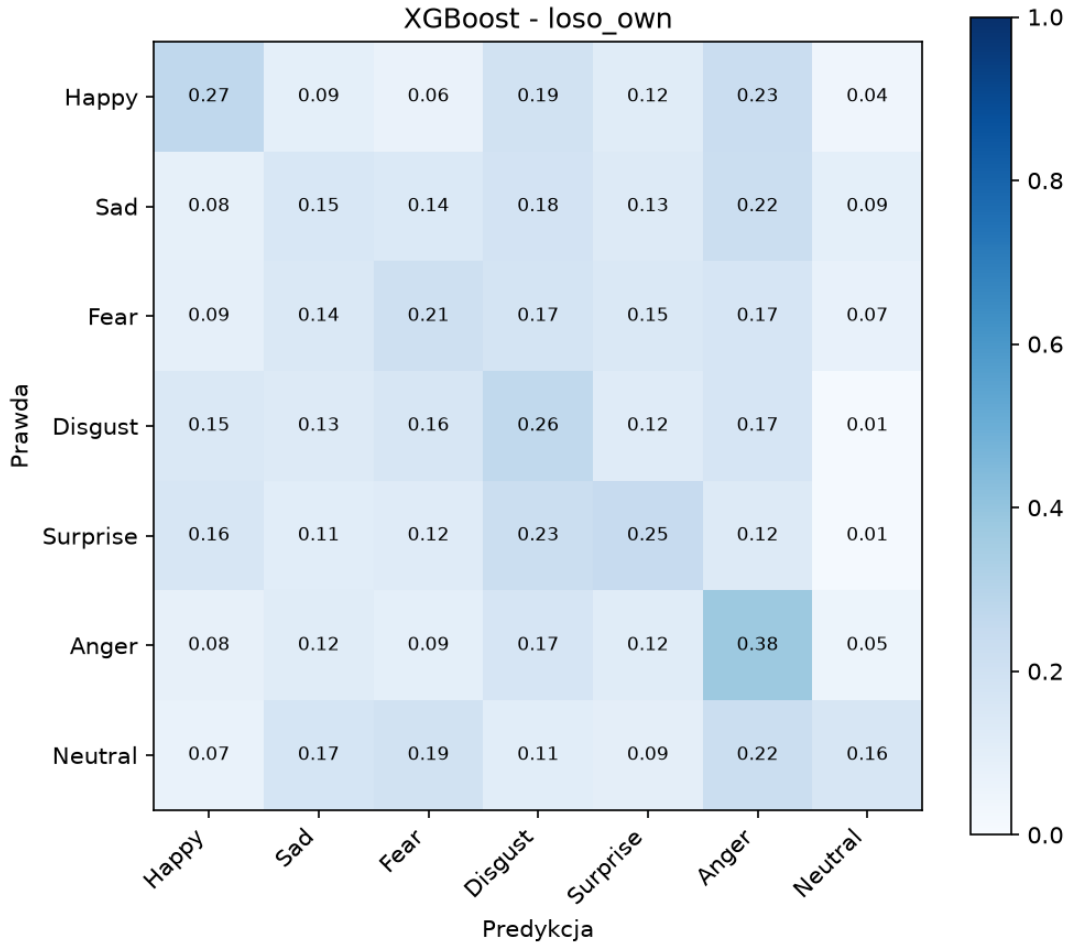
Rozpatrując wariant własny - najwyższe macro-F1 w walidacji subject-dependent uzyskał klasyfikator SVM (**0,4097**), przy accuracy na poziomie **42,06 %**. Dla LOSO najwyższe macro-F1 uzyskał XGBoost (**0,2371**), gdzie accuracy wynosi **24,25 %**. Wyniki klasyfikatorów zależały więc od sposobu wydzielania danych testowych. W przypadku stimulus-independent wyniki macro-F1 SVM i XGBoost były zbliżone - **0,3797** i **0,3786**. W LOSO model XGBoost przewyższał MostFrequent o **9,25 punktu procentowego** accuracy.

Dla klasyfikatora SVM przy wykorzystaniu cech własnych accuracy zmniejszyło się z 42,06 % w subject-dependent do 23,69 % w LOSO, co daje **18,37 punktu procentowego**. W przypadku cech referencyjnych dla analogicznej zmiany różnica wyniosła **8,25 punktu procentowego** (z 30,31 % do 22,06 %). Widoczne obniżone wyniki w LOSO wskazują na trudności w klasyfikowaniu danych osoby nieobecnej w zbiorze treningowym w porównaniu do sytuacji, gdy dane tych samych uczestników mogły występować w obu częściach - treningowej i testowej.

Własna reprezentacja cech uzyskała wartości accuracy i macro-F1 wyższe niż cechy referencyjne we wszystkich dziewięciu porównaniach. Warto zaznaczyć, że nie jest to jednoznaczny dowód na uniwersalność tej przewagi, a różnica zaobserwowana dla tej analizy. Wynika to z ograniczeń spowodowanych dostępem do jednego zbioru danych, co nie pozwala uogólnić tego wniosku na inne zbiory i zastosowania.

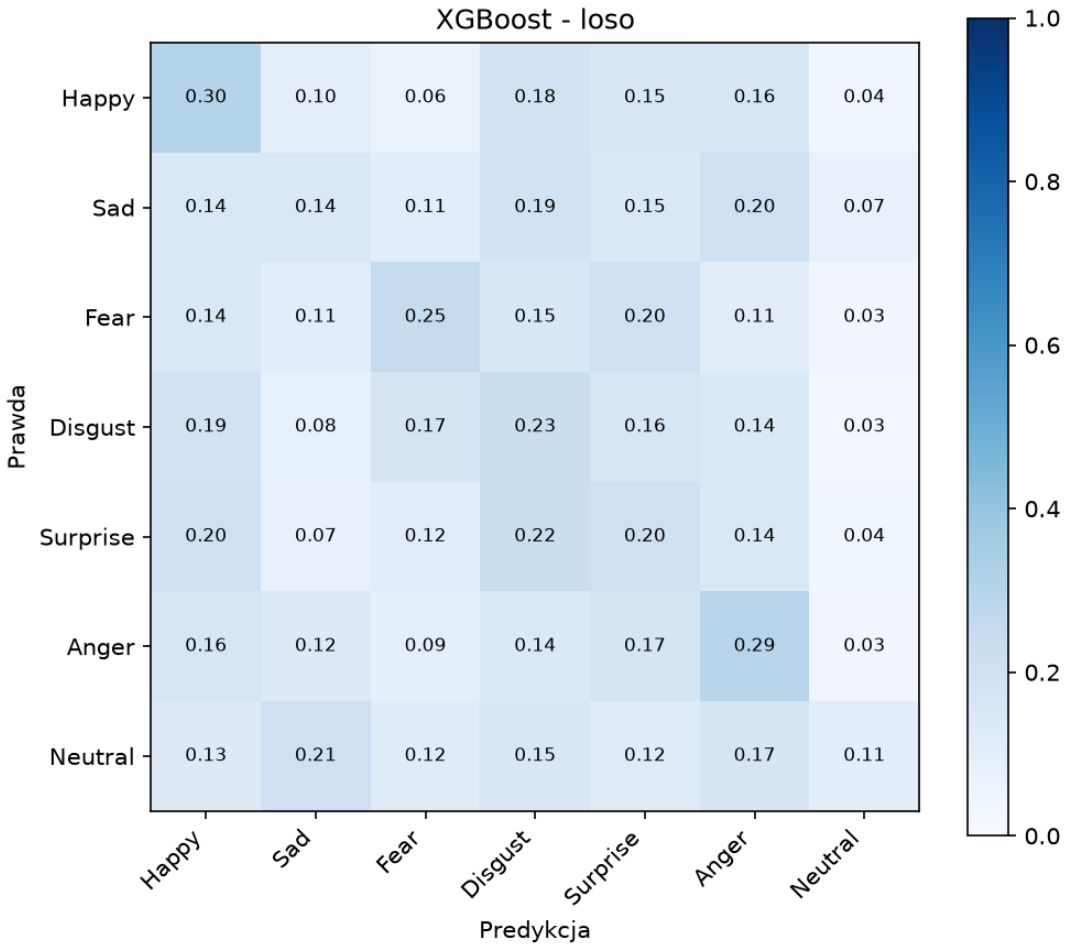
4.3 Macierz pomyłek

Macierz pomyłek modelu XGBoost w walidacji LOSO dla cech własnych przedstawiono na rysunku 4.1, a dla cech referencyjnych na rysunku 4.2. Wartości czułości dla poszczególnych klas zestawiono w tabeli 4.4. Pogrubieniem oznaczono wyższą wartość recall dla każdej klasy.



Rysunek 4.1. Macierz pomyłek XGBoost w LOSO - cechy własne

Analiza z użyciem macierzy pomyłek pomoże zidentyfikować, które klasy zostały poprawnie rozpoznane przez model - mówi o tym kolumna czułości dla danej klasy (recall). Dla XGBoost z cechami własnymi najwyższy recall w LOSO dotyczył złości (**37,50 %**), a najniższy - smutku (**15,42 %**). W przypadku cech referencyjnych - recall radości i strachu był wyższy.



Rysunek 4.2. Macierz pomyłek XGBoost w LOSO - cechy referencyjne

Tabela 4.4. Czułość (ang. recall) klasyfikatora XGBoost w LOSO dla poszczególnych klas

Klasa	Recall referencyjny XGBoost LOSO [%]	Recall własny XGBoost LOSO [%]
Happy - radość	**30,42**	26,67
Sad - smutek	14,17	**15,42**
Fear - strach	**24,58**	20,83
Disgust - wstręt	22,50	**26,25**
Surprise - zaskoczenie	20,42	**24,58**
Anger - złość	29,17	**37,50**
Neutral - stan neutralny	10,63	**15,63**

4.4 Wiarygodność wyników

Tabela 4.5. Trafność i metryki probabilistyczne modeli w LOSO

Wariant	Model w LOSO	Accuracy [%]	Średnie maksimum *predict_proba* [%]	Log-loss	Brier	ECE
Referencyjny	SVM	22,06	29,54	1,9485	0,8539	0,0748
Referencyjny	Random Forest	21,94	23,40	**1,9065**	**0,8455**	**0,0146**
Referencyjny	XGBoost	**22,25**	52,61	2,3806	0,9849	0,3036
Własny	SVM	23,69	36,96	2,0520	0,8732	0,1328
Własny	Random Forest	22,63	25,19	**1,8819**	**0,8381**	**0,0257**
Własny	XGBoost	**24,25**	58,84	2,5015	1,0154	0,3459

Tabela 4.5 przedstawia trafność oraz metryki probabilistyczne modeli w walidacji LOSO. Kolumna Accuracy odnosi się do odsetka poprawnie sklasyfikowanych prób, "średnie maksimum *predict_proba*" jest średnią najwyższych wartości wyświetlanych przy predykcji. Log-loss ocenia jakie prawdopodobieństwo model przypisał prawdziwej klasie - im niższa wartość log-loss, tym korzystniejszy wynik. Brier porównuje wektor siedmiu wartości (klas emocji) *predict_proba* z prawdziwą klasą. Również dla tej metryki mniejszy wynik jest lepszy - 0 oznacza wynik idealny. Natomiast ECE mierzy zgodność pomiędzy najwyższymi wartościami *predict_proba*, a tym jak często obserwowano poprawne odpowiedzi [45, 50]. Pogrubiono najwyższe accuracy oraz najniższe wartości log-loss, Brier i ECE osobno dla każdego wariantu.

Mamy więc nie tylko trafność klasyfikacji, a także jakość zwracanych wartości przez metodę *predict_proba*. Dla obu testowanych wariantów, najwyższą wartość accuracy uzyskał model XGBoost. Mimo to wartości log-loss, Brier score oraz ECE były w tym przypadku najwyższe. Średnia największa wartość *predict_proba* wynosiła ponad 50 %, a dla wariantu własnego aż **58,84 %**. Oznacza to, że model wykazuje nadmierną pewność odpowiedzi.

Random Forest mimo niższego accuracy uzyskiwał w obu wariantach wartości *predict_proba*, które świadczą o najlepszej zgodności deklarowanej pewności z obserwowaną trafnością. Świadczą o tym także najniższe w obrębie wariantów wyniki ECE: **0,0146** dla wariantu referencyjnego i **0,0257** dla wariantu własnego.

Podsumowując wyniki dochodzimy do wniosku, że wyższe macro-F1, czy accuracy wcale nie musi oznaczać, że model generuje bardziej wiarygodne wartości probabilistyczne.

4.5 Porównanie literaturowe modeli

Aby określić położenie wyników osiąganych przez modele w niniejszej pracy, porównano je z innymi badaniami, które również korzystały z bazy danych "SEED-VII". W tabelach 4.6-4.8 pogrubiono modele oraz najwyższe raportowane wartości accuracy, a w tabeli 4.6 również najwyższą wartość macro-F1.

1) Badanie Z. Zhou i in. [13]
   W porównaniu uwzględniono między innymi wielowarstwowy perceptron (ang. multilayer perceptron, MLP) oraz multimodalny adaptacyjny transformator emocji (ang. multimodal adaptive emotion transformer, MAET).

Tabela 4.6. Porównanie wyników z badaniem [13]

Badanie	Model	Accuracy [%]	Macro-F1 [%]	Różnica accuracy względem XGBoost niniejszej pracy
Niniejsza praca	XGBoost, cechy własne	24,25	23,71	—
Z. Zhou i in.	Random Forest	20,4 ± 4,7	14,0 ± 5,9	+3,85 p.p.
Z. Zhou i in.	Logistic Regression	21,0 ± 4,4	14,9 ± 5,6	+3,25 p.p.
Z. Zhou i in.	SVM	21,7 ± 4,2	15,8 ± 5,7	+2,55 p.p.
Z. Zhou i in.	MLP	22,5 ± 5,2	16,6 ± 6,3	+1,75 p.p.
Z. Zhou i in.	MAET, 310 cech DE	36,4	24,2	−12,15 p.p.
Z. Zhou i in.	**MAET, cechy lokalne i globalne**	**40,1**	**38,7**	−15,85 p.p.

Porównując model XGBoost wykorzystujący cechy własne niniejszej pracy z modelami opisanymi w pracy [13] można wywnioskować, że dla klasycznych modeli, takich jak SVM, Random Forest, regresji logistycznej, czy MLP ma konkurencyjne wyniki. Dopiero przy wyspecjalizowanych modelach takich jak MAET można zaobserwować znacznie gorszy wynik. Największa różnica accuracy na niekorzyść modelu XGBoost przypada na model wykorzystujący połączenie cech DE, cech grafowych, a także globalnych właściwości sygnału. Pełne porównanie tych wyników z modelem XGBoost wykorzystującym cechy własne przedstawiono w tabeli 4.6.

Model wypada dobrze na tle tradycyjnych metod lecz dodatkowe cechy i wyspecjalizowana architektura neuronowa pozwalają osiągać lepsze wyniki. Warte zaznaczenia jest to, że wyniki pochodzą z oddzielnych eksperymentów - różnią się zestawem cech, normalizacją, czy agregacją metryk. Z tego powodu porównania nie należy traktować jako dowodu przewagi statystycznej, ma ono charakter orientacyjny.

2) Badanie [10]

W zestawieniu występują: model k najbliższych sąsiadów (ang. k-nearest neighbors, KNN), regularyzowana grafowa sieć neuronowa (ang. regularized graph neural network, RGNN), grafowa sieć konwolucyjna z mechanizmem uwagi kanałowej (ang. graph convolutional network with channel attention, GCNCA), hierarchiczna konwolucyjna sieć neuronowa (ang. hierarchical convolutional neural network, HCNN), model transformatorowy (ang. Transformer) oraz wcześniej opisany MAET. Zestawienie wyników przedstawiono w tabeli 4.7.

Tabela 4.7. Porównanie wyników z badaniem [10]

Model	Accuracy [%]	Różnica względem XGBoost niniejszej pracy
KNN	20,85 ± 4,56	+3,40 p.p.
RGNN	37,49 ± 5,44	−13,24 p.p.
GCNCA	38,68 ± 3,94	−14,43 p.p.
HCNN	39,88 ± 4,94	−15,63 p.p.
Transformer	40,36 ± 5,22	−16,11 p.p.
MAET	**40,90 ± 5,52**	−16,65 p.p.
XGBoost niniejszej pracy	24,25	—

W pracy [10] do klasyfikacji wykorzystano czterosekundowe segmenty EEG, w przeciwieństwie do niniejszej pracy, gdzie jednostką predykcji jest cała próba. Z tego powodu porównanie również należy traktować jako kontekst literaturowy. Najprostszy model przedstawiony w publikacji - KNN osiąga accuracy niższe niż porównywany XGBoost. Pozostałe modele to architektury o wiele bardziej złożone, co powoduje, że znacząco odbiegają poziomem accuracy.

3) Badanie [14]

W zestawieniu uwzględniono kompaktową konwolucyjną sieć neuronową do zastosowań EEG (ang. EEGNet), konwolucyjny transformator EEG (ang. EEG Conformer), lekką wielowymiarową sieć uwagi (ang. lightweight multi-dimensional attention network, LMDA), sieć konwolucyjną i transformatorową z mieszanym mechanizmem uwagi (ang. mixed attention-based convolution and transformer network, MACTN) oraz uczenie hierarchiczne inspirowane procesami poznawczymi (ang. cognitive-inspired hierarchical learning, CIHL). Porównanie ich wyników z modelem XGBoost wykorzystującym cechy własne zestawione zostało w tabeli 4.8.

W powyższym badaniu LOSO na danych SEED-VII otrzymano wyniki accuracy na poziomie zbliżonym do XGBoost niniejszej pracy. Wymienione architektury są modelami głębokiego uczenia, automatycznie wyodrębniające przestrzenne i czasowe właściwo-

Tabela 4.8. Porównanie wyników z innymi modelami [14]

Model	Accuracy [%]	Różnica względem XGBoost niniejszej pracy
EEGNet	21,85	+2,40 p.p.
EEG-Conformer	22,72	+1,53 p.p.
LMDA	24,31	−0,06 p.p.
MACTN	23,37	+0,88 p.p.
CIHL	**26,13**	−1,88 p.p.

ści EEG. Można więc wywnioskować, że w niektórych przypadkach porównywany klasyfikator, który wykorzystuje zagregowane cechy DE, osiąga bardzo zbliżone rezultaty do złożonych architektur neuronowych. Wciąż jednak należy mieć na uwadze, że badania różnią się od siebie na różnych etapach, więc wyniki mają charakter poglądowy.

4.6 Weryfikacja aplikacji

Aby zweryfikować działanie aplikacji wykonano testy automatyczne oraz ręczne - na podstawie przygotowanych scenariuszy. Weryfikacja działania aplikacji ma na celu potwierdzenie poprawności działania elementów funkcjonalnych - nie świadczy o skuteczności modelu. Jakość klasyfikacji została oceniona za pomocą walidacji w poprzednich podrozdziałach.

4.6.1 Testy automatyczne

W projekcie zastosowano testy pytest, których zadaniem jest sprawdzenie rozpoznawania sygnału i cech, normalizacji orientacji tablic, tworzenia wektora 310 cech, obliczania DE oraz wygładzania LDS, mapowania numerów prób na emocje, rozłączności bodźców w stimulus-independent, odrzucania wartości takich jak: NaN, nieskończoności, PSD i danych okulograficznych oznaczonych jako EYE (ang. eye movement features), kontrolowania częstotliwości próbkowania i nazw kanałów, generowania plików MAT oraz PDF, czy powtarzalności funkcji predykcji.

Przykładowe testy aplikacji:

1) Rozpoznanie formatu danych - listing 4.1.

Listing 4.1. Test rozpoznawania formatu sygnału i cech

```
def test_rozpoznawanie_formatu():
    features = np.zeros((15, 5, 62))
    signal = np.random.default_rng(42).normal(
        0, 15, size=(62, 12000)
    )

    assert detect_format(features) == "features"
    assert detect_format(signal) == "signal"
```

Test sprawdza, czy aplikacja rozróżnia gotowe cechy o wymiarach (15, 5, 62) oraz sygnał EEG zawierający 62 kanały.

2) Przygotowanie wektora modelu - listing 4.2.

Listing 4.2. Test wymiarów wektora i sekwencji cech

```
def test_przygotowanie_wektora_cech():
    features = np.ones((15, 5, 62))

    vector, feature_sequence = prepare_feature_vector(
        features, "features"
    )

    assert vector.shape == (310,)
    assert feature_sequence.shape == (15, 5, 62)
```

Test służy do potwierdzenia, że sekwencja cech zostaje zagregowana do wektora 310 wartości, zgodnego z wejściem klasyfikatora.

3) Odrzucenie nieprawidłowego wejścia - listing 4.3.

Listing 4.3. Test odrzucania sygnału o nieprawidłowej liczbie kanałów

```
def test_nieprawidlowa_liczba_kanalow():
    invalid_signal = np.zeros((61, 1600))

    with pytest.raises(MatFileError):
        prepare_feature_vector(invalid_signal, "signal")
```

Test sprawdza działanie walidacji na sytuację gdy sygnał zawiera 61 zamiast 62 kanały. W tym przypadku oczekiwany rezultat to zgłoszenie kontrolowanego błędu MatFileError.

4) Rozłączność bodźców w stimulus-independent - listing 4.4.

Listing 4.4. Test rozłączności bodźców między treningiem a testem w walidacji SI

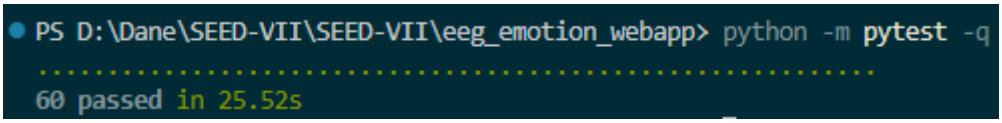
```
def test_stimulus_independent_nie_miesza_bodzcow():
      trials = np.repeat(np.arange(35), 3)
      y = np.repeat(np.tile(np.arange(7), 5), 3)
      subjects = np.tile(np.arange(3), 35)
      X = np.zeros((len(y), 1))

      cv, groups, _ = validation_split(
          "stimulus_independent", subjects, trials
      )

      for train_index, test_index in cv.split(X, y, groups):
          assert set(trials[train_index]).isdisjoint(
              trials[test_index]
          )
```

Test potwierdza, że w walidacji stimulus-independent identyfikatory bodźców znajdujące się w części testowej nie występują w części treningowej.

Ze względu na dużą liczbę testów przedstawiono jedynie kilka przykładów ilustrujących różne obszary weryfikacji. Testy uruchamia polecenie python-mpytest-q, a wynik jego wykonania pokazano na rysunku 4.3.



Rysunek 4.3. Wynik uruchomienia testów automatycznych

Wykonano łącznie **60 testów**, **wszystkie zakończyły się powodzeniem**.

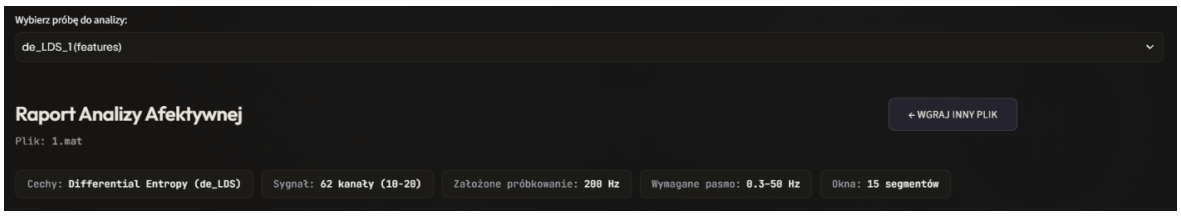
4.6.2 Ręczne testy

Scenariusze testów ręcznych oraz uzyskane wyniki zestawiono w tabeli 4.9.

Tabela 4.9. Scenariusze i wyniki testów ręcznych

ID	Scenariusz	Oczekiwany rezultat	Wynik działania
TEST01	Wczytanie prawidłowego pliku z cechami de_LDS	Rozpoznanie formatu cech i użycie modelu referencyjnego	**Pozytywny**
TEST02	Wczytanie prawidłowego sygnału EEG	Rozpoznanie sygnału, ekstrakcja 310 cech i użycie modelu własnej ekstrakcji	**Pozytywny**
TEST03	Plik zawierający kilka prób	Wyświetlenie listy prób i możliwość wyboru jednej z nich	**Pozytywny**
TEST04	Wykonanie klasyfikacji	Wyświetlenie jednej klasy oraz siedmiu wartości *predict_proba*	**Pozytywny**
TEST05	Powtórna analiza tego samego pliku	Uzyskanie identycznego wyniku	**Pozytywny**
TEST06	Zmiana fragmentu suwakiem	Zmiana prezentowanego fragmentu bez zmiany predykcji dla całej próby	**Pozytywny**
TEST07	Pobranie raportu PDF	Utworzenie poprawnego pliku, który można otworzyć	**Pozytywny**
TEST08	Eksport fragmentu do MAT	Utworzenie pliku możliwego do ponownego odczytania	**Pozytywny**
TEST09	Wczytanie cech PSD albo danych EYE	Odrzucenie nieobsługiwanego rodzaju danych	**Pozytywny**
TEST10	Wybranie analizy kolejnego pliku	Wyczyszczenie poprzedniego stanu aplikacji	**Pozytywny**

Wszystkie testy przewidziane dla scenariusza zostały zakończone **wynikiem pozytywnym**. Przykład wczytania pliku zawierającego referencyjne cechy de_LDS i wyboru próby pokazano na rysunku 4.4. Wpływ zmiany zakresu czasu na prezentowany fragment sygnału pokazano na rysunku 4.5, natomiast reakcję aplikacji na próbę wczytania nieprawidłowych danych obrazuje rysunek 4.6.



Rysunek 4.4. Wczytanie pliku zawierającego referencyjne cechy de_LDS i wybór próby do analizy.



Rysunek 4.5. Wpływ zmiany czasu na prezentowany fragment sygnału przy wykorzystaniu suwaka - Zakresy od 0 do 10 sekundy oraz od 20 do 30 sekundy.



Rysunek 4.6. Przykładowa reakcja aplikacji na wgranie nieprawidłowych danych w postaci stosownego komunikatu błędu. W tym przypadku danymi wejściowymi był plik zawierający cechy okulograficzne.

5. Podsumowanie i wnioski

Celem pracy było stworzenie aplikacji webowej, która umożliwi analizę sygnałów EEG z użyciem modelu uczenia maszynowego. Została zaimplementowana możliwość obsługi zarówno sygnałów EEG, jak i gotowych cech wykorzystując najlepsze dla wybranego wejścia modele. Zastosowane funkcjonalności i integracje, tworzą spójny prototyp badawczy.

5.1 Ocena realizacji celu i rezultatów pracy

Założony cel pracy został osiągnięty. Aplikacja obsługuje dane wejściowe zgodne z opisaną w rozdziale trzecim strukturą, rozpoznaje ich format, wykonuje ekstrakcję oraz agregację cech. Przeprowadza predykcję przedstawiając wyniki klasyfikacji siedmiu klas emocji. Przeprowadzono testy, które miały na celu sprawdzenie poprawności działania najważniejszych operacji, funkcjonalności oraz ograniczeń nałożonych na system.

Wyniki analizy modeli wskazują na dużą zależność jakości klasyfikacji od sposobu wydzielenia danych testowych. Najwyższe rezultaty uzyskano dla walidacji subject-dependent, a najniższe - LOSO. Dla cech własnych accuracy SVM zmniejszyło się z 42,06 % w subject dependent do 23,69 % w LOSO. Model korzystał więc w znacznym stopniu z właściwości charakterystycznych dla uczestników obecnych w zbiorze treningowym. Przeniesienie predykcji na nową osobę okazuje się znacznie trudniejsze.

W rozdziale czwartym skupiono się głównie na walidacji LOSO, gdyż jest najbliższa realnym doświadczeniom dla nowego użytkownika, który analizuje dane niezwiązane z bazą treningową. Najwyższy wynik uzyskał w niej model XGBoost wykorzystujący cechy własne - 24,25 % accuracy i 0,2371 macro-F1. Model przewyższył bazowy MostFrequent o 9,25 punktu procentowego - mimo to wciąż błędnie klasyfikuje większość prób. Macierz pomyłek pokazała różnice między sklasyfikowanymi klasami. Najwyższy recall uzyskano dla złości, natomiast najniższy dla smutku.

Wariant własny osiągał wyższe wyniki niż referencyjny we wszystkich przeprowadzonych porównaniach dla właściwych klasyfikatorów. Można więc wywnioskować, że zastosowana implementacja ekstrakcji cech okazała się być przydatna i pozytywnie wpływa na otrzymywane wyniki klasyfikacji. Nie można jednak stwierdzić, że jest to cecha uniwersalna, gdyż to wymagałoby wykonania badania oddzielającego wpływ filtracji, sposobu obliczania DE, wygładzania LDS i agregacji czasowej.

Analiza wartości zwracanych przez aplikację poprzez metodę *predict_proba* wskazuje, że otrzymywane predykcje nie powinny być interpretowane jako wierne odzwierciedlenie stanu faktycznego dla klasy badanego sygnału - szczególnie w przypadku danych spoza bazy treningowej.

Uzyskana trafność była niska. Wyniki mieściły się jednak w zakresie części przytoczonych rezultatów literaturowych. Biorąc pod uwagę ilość klas, czy ograniczenia związane z dostępnymi zasobami, wynik można uznać za zadowalający.

Podsumowując, wykonana aplikacja spełnia założenia funkcjonalne, przechodząc przygotowane testy bezbłędnie. System demonstruje kompletny proces przetwarzania i klasyfikacji danych EEG, lecz w związku z obecnymi ograniczeniami, które obniżają jego skuteczność - nie pozwala na wystarczająco wiarygodne wyniki w zastosowaniu praktycznym. Projekt jest jednak świetnym punktem wyjścia do przeprowadzania dalszych badań, rozwoju i pracy nad lepszymi rozwiązaniami w celu uzyskiwania coraz lepszych wyników.

5.2 Ograniczenia

W trakcie tworzenia systemu, pozyskiwania zbiorów danych i testów, napotkano ograniczenia, które wpływają na końcowy wynik i działanie aplikacji. Autor posiadał dostęp do jednego profesjonalnego zbioru - SEED-VII, który został udostępniony przez twórców do wykorzystania w celach naukowych. Obejmuje on 20 uczestników i 1600 prób, a dane zarejestrowano w kontrolowanych warunkach laboratoryjnych - przy użyciu tego samego rodzaju aparatury, takiego samego rozmieszczenia elektrod i według identycznego przebiegu eksperymentu. Nie sprawdzono więc, jak działałby model na danych pochodzących z innego urządzenia lub zebranych w innych warunkach. Uzyskane wyniki odnoszą się przede wszystkim do danych przygotowanych w sposób zgodny ze zbiorem SEED-VII.

Wpłynęło to zdecydowanie na rozdział dotyczący analizy. Gdyby dostęp do innych zbiorów był mniej ograniczony, warto byłoby przeprowadzić na nich zewnętrzną walidację. Z tych względów aplikacja ma głównie charakter badawczy.

5.3 Kierunki dalszych prac

Aplikacja ma duży potencjał na rozwój. Warto w przyszłości spróbować zachować czasową strukturę sygnału. Obecnie cechy ze wszystkich okien jednej próby są uśrednione, usuwając informacje o kolejności i dynamice reakcji. Aby umożliwić analizę zmian, jakie zachodzą w czasie całej próby, można zastosować modele sekwencyjne, takie jak: długa pamięć krótkotrwała (ang. long short-term memory, LSTM), bramkowana jednostka rekurencyjna (ang. gated recurrent unit, GRU) czy EEG-Conformer.

Szczególnej uwagi wymaga kalibracja modelu opierająca się na próbach nowego użytkownika i analiza wyników na niezależnym zbiorze danych. Dużą wartość dodaną do badań wprowadziłoby przeprowadzenie badań pozwalających osobno ocenić wpływ wygładzania LDS, długości okna, agregacji czasowej i rodzaju cech.

Aplikację można rozwinąć, dodając obsługę strumienia EEG w czasie rzeczywistym z automatycznym wykrywaniem artefaktów. Aby było to jednak możliwe, najpierw trzeba upewnić się, że system będzie poprawnie działał na danych wcześniej już zarejestrowanych, a predykcje modeli będą o wiele bardziej wiarygodne. Polepszając wyniki i rozbudowując funkcjonalności, aplikacja mogłaby stać się w pełni użytkowym narzędziem.

Jednym z możliwych kierunków rozwoju aplikacji jest również wykorzystanie danych dotyczących ruchu oczu udostępnionych w zbiorze SEED-VII. Cechy zawarte w katalogu EYE_features, związane między innymi ze średnicą źrenicy, fiksacjami, sakkadami i mrugnięciami, mogłyby zostać połączone z cechami EEG w ramach klasyfikacji multimodalnej. Informacje pochodzące z obu źródeł mogą się wzajemnie uzupełniać, a tym samym potencjalnie zwiększać skuteczność rozpoznawania emocji [10]. Wymagałoby to jednak synchronizacji obu rodzajów danych, rozszerzenia formatu wejściowego aplikacji oraz ponownego wytrenowania i porównania modeli dla danych EEG, danych okulograficznych i ich połączenia.

Spis rysunków

2.1 Mechanizm przekazywania sygnału na poziomie pojedynczej synapsy [11] . 5
2.2 Przykładowe dane EEG [10, 43] 6
2.3 Rozmieszczenie elektrod w systemie 10–20 [12] 7

3.1 Przepływ danych i główne funkcje aplikacji [43] 18
3.2 Przykładowa zawartość pliku przed i po ekstrakcji cech [10, 43] 22
3.3 Ogólny cykl przetwarzania danych wejściowych [43] 24
3.4 Struktura pakietu ml 36
3.5 Struktura pakietu app 36
3.6 Przepływ przygotowania modelu i analizy pliku wejściowego [43] 37
3.7 Proces interakcji użytkownika z aplikacją [43] 40
3.8 Interfejs aplikacji - strona startowa 41
3.9 Interfejs aplikacji - klasyfikacje emocji 41
3.10 Interfejs aplikacji - wybrany sygnał próby 41

4.1 Macierz pomyłek XGBoost w LOSO - cechy własne 46
4.2 Macierz pomyłek XGBoost w LOSO - cechy referencyjne 47
4.3 Wynik uruchomienia testów automatycznych 53
4.4 Wczytanie pliku zawierającego referencyjne cechy de_LDS i wybór próby do analizy. 54
4.5 Wpływ zmiany czasu na prezentowany fragment sygnału przy wykorzystaniu suwaka - Zakresy od 0 do 10 sekundy oraz od 20 do 30 sekundy. 55
4.6 Przykładowa reakcja aplikacji na wgranie nieprawidłowych danych w postaci stosownego komunikatu błędu. W tym przypadku danymi wejściowymi był plik zawierający cechy okulograficzne. 55

Spis tabel

2.1 Pasma częstotliwości sygnału EEG [1, 4, 5] . . . 3
2.2 Oznaczenia lokalizacji elektrod [12] . . . 7
2.3 Rodzaje elektrod [1] . . . 8
2.4 Wykorzystane technologie, biblioteki i frameworki . . . 16

3.1 Wymagania funkcjonalne aplikacji . . . 19
3.2 Wymagania dotyczące danych wejściowych . . . 19
3.3 Charakterystyka zbioru SEED-VII [10] . . . 20
3.4 Klasy emocji i liczba obserwacji [10] . . . 21
3.5 Struktura plików danych [10] . . . 22
3.6 Pasma częstotliwości [1, 10] . . . 25
3.7 Modele i parametry zastosowane w implementacji . . . 27
3.8 Porównanie sposobów walidacji . . . 28
3.9 Moduły aplikacji i ich odpowiedzialności . . . 37

4.1 Podział danych w poszczególnych walidacjach . . . 44
4.2 Wyniki klasyfikacji - wariant referencyjny . . . 44
4.3 Wyniki klasyfikacji - wariant własny . . . 45
4.4 Czułość (ang. recall) klasyfikatora XGBoost w LOSO dla poszczególnych klas 47
4.5 Trafność i metryki probabilistyczne modeli w LOSO . . . 48
4.6 Porównanie wyników z badaniem [13] . . . 49
4.7 Porównanie wyników z badaniem [10] . . . 50
4.8 Porównanie wyników z innymi modelami [14] . . . 51
4.9 Scenariusze i wyniki testów ręcznych . . . 54

Spis listingów

3.1 Mapowanie numeru próby na kategorię emocjonalną i etykietę klasy 23
3.2 Filtracja pasmowa i obliczanie DE w oknach czasowych 25
3.3 Obliczanie DE na podstawie wariancji sygnału 26
3.4 Agregacja cech próby do wektora 310 wartości 26
3.5 Obliczanie accuracy i macro-F1 29
3.6 Wyznaczanie i zapis wykresu macierzy pomyłek 30
3.7 Normalizacja wierszy macierzy pomyłek 30
3.8 Obliczanie i zapis accuracy dla poszczególnych osób lub bodźców 30
3.9 Wizualizacja accuracy poszczególnych osób w LOSO 31
3.10 Wyznaczanie przedziału ufności accuracy metodą bootstrapu grup 31
3.11 Obliczanie straty logarytmicznej (log-loss) 31
3.12 Kodowanie etykiet klas w reprezentacji one-hot 32
3.13 Obliczanie wieloklasowej miary Brier 32
3.14 Obliczanie oczekiwanego błędu kalibracji (ECE) 33
3.15 Definicja modelu bazowego MostFrequent 33
3.16 Definicja losowego modelu bazowego Stratified 34
3.17 Wyznaczanie etykiet klas na podstawie predykcji OOF 34
3.18 Wyznaczanie prawdopodobieństw klas dla wektora cech wejściowych 34
3.19 Wybór klasy o najwyższym przewidywanym prawdopodobieństwie 35
3.20 Struktura artefaktu modelu i jego metadanych 35
3.21 Wczytywanie tablic danych z pliku MAT 38
3.22 Rozpoznawanie formatu danych i walidacja sygnału lub cech 38
3.23 Dobór artefaktu modelu do formatu danych wejściowych 39
3.24 Obsługa braku artefaktu modelu 39
3.25 Inicjalizacja stanu sesji aplikacji 42
3.26 Zapamiętywanie danych wczytanego pliku w stanie sesji 42
3.27 Resetowanie danych wejściowych i odświeżanie interfejsu 42
3.28 Wczytywanie i buforowanie artefaktu modelu 42
4.1 Test rozpoznawania formatu sygnału i cech 52
4.2 Test wymiarów wektora i sekwencji cech 52
4.3 Test odrzucania sygnału o nieprawidłowej liczbie kanałów 52
4.4 Test rozłączności bodźców między treningiem a testem w walidacji SI 53

Bibliografia

[1] M. Teplan, “Fundamentals of EEG measurement,” *Measurement Science Review*, vol. 2, no. 2, pp. 1-11, 2002.

[2] M. Gruber, “Proofs of the Nyquist-Shannon sampling theorem,” Bachelor’s thesis, Dept. Math. Stat., Univ. Konstanz, Konstanz, Germany, 2013.

[3] H. J. Scheer, T. Sander, and L. Trahms, “The influence of amplifier, interface and biological noise on signal quality in high-resolution EEG recordings,” *Physiological Measurement*, vol. 27, no. 2, pp. 109-117, Feb. 2006.

[4] S. M. Alarcão and M. J. Fonseca, “Emotions recognition using EEG signals: A survey,” *IEEE Transactions on Affective Computing*, vol. 10, no. 3, pp. 374-393, Jul.-Sep. 2019.

[5] L. Hu and Z. Zhang, Eds., *EEG Signal Processing and Feature Extraction*. Singapore: Springer Nature, 2019.

[6] F. H. Lopes da Silva, “EEG analysis: Theory and practice,” in *Electroencephalography: Basic Principles, Clinical Applications, and Related Fields*, 5th ed., E. Niedermeyer and F. H. Lopes da Silva, Eds. Philadelphia, PA, USA: Lippincott Williams & Wilkins, 2005, pp. 1225-1255.

[7] T. Uktveris and V. Jusas, “Development of a modular board for EEG signal acquisition,” *Sensors*, vol. 18, no. 7, Art. no. 2140, Jul. 2018.

[8] W.-L. Zheng and B.-L. Lu, “Investigating critical frequency bands and channels for EEG-based emotion recognition with deep neural networks,” *IEEE Transactions on Autonomous Mental Development*, vol. 7, no. 3, pp. 162-175, Sep. 2015.

[9] I. Winkler, S. Haufe, and M. Tangermann, “Automatic classification of artifactual ICA components for artifact removal in EEG signals,” *Behavioral and Brain Functions*, vol. 7, no. 1, Art. no. 30, Aug. 2011.

[10] W.-B. Jiang, X.-H. Liu, W.-L. Zheng, and B.-L. Lu, “SEED-VII: A multimodal dataset of six basic emotions with continuous labels for emotion recognition,” *IEEE Transactions on Affective Computing*, vol. 16, no. 2, pp. 969-985, 2025, doi: 10.1109/TAFFC.2024.3485057.

[11] S. K. Votis, “Postsynaptic potentials,” *Kenhub*. [Online]. Available: https://www.kenhub.com/en/library/physiology/postsynaptic-potentials.

[12] A. Bielskis, V. Denisov, G. Kucinskas, O. Ramašauskas, and N. Romas, “Modeling of human physiological parameters in an e-laboratory by SOM neural networks,” *Elektronika ir Elektrotechnika*, vol. 75, pp. 77-82, Mar. 2007, doi: 10.5755/j02.eie.10704.

[13] Z. Zhou, I. McEvoy, and C. E. Valderrama, “Local-global feature fusion for subject-independent EEG emotion recognition,” arXiv:2601.08094v1 [cs.LG], Jan. 13, 2026. [Online]. Available: https://arxiv.org/abs/2601.08094v1.

[14] W. Han, H. Weng, Z. Liao, Z. Xu, and C. Chen, “Cognitive-inspired hierarchical learning framework for cross-dataset EEG-based emotion recognition,” *Electronics*, vol. 15, no. 10, Art. no. 1971, 2026, doi: 10.3390/electronics15101971.

[15] Analog Devices, “An introduction to electrical filters,” *Analog Devices Wiki*. [Online]. Available: https://wiki.analog.com/university/courses/engineering_discovery/lab_5.

[16] S. Puthusserypady, “Digital systems,” in *Applied Signal Processing*. Now Publishers, 2021, doi: 10.1561/978-1-68083-979-120251011.

[17] EEGLAB, “Independent component analysis for artifact removal,” *EEGLAB Wiki*. [Online]. Available: https://eeglab.org/tutorials/06_RejectArtifacts/RunICA.html.

[18] G. Buzsáki, C. A. Anastassiou, and C. Koch, “The origin of extracellular fields and currents-EEG, ECoG, LFP and spikes,” *Nature Reviews Neuroscience*, vol. 13, no. 6, pp. 407-420, Jun. 2012, doi: 10.1038/nrn3241.

[19] W. O. Tatum et al., “Clinical utility of EEG in evaluating patients with suspected epilepsy,” *Epilepsia*, vol. 59, no. 6, pp. 1056-1073, Jun. 2018, doi: 10.1111/epi.14057.

[20] C. Babiloni et al., “Early diagnosis of Alzheimer’s disease: The role of biomarkers including advanced EEG signal analysis. Report from the IFCN-sponsored panel of experts,” *Clinical Neurophysiology*, vol. 131, no. 6, pp. 1287-1310, Jun. 2020, doi: 10.1016/j.clinph.2020.03.003.

[21] S. K. Loo and S. Makeig, “Clinical utility of EEG in attention-deficit/hyperactivity disorder: A research update,” *Neurotherapeutics*, vol. 9, no. 3, pp. 569-587, Jul. 2012, doi: 10.1007/s13311-012-0131-z.

[22] S. Koelstra et al., “DEAP: A database for emotion analysis using physiological signals,” *IEEE Transactions on Affective Computing*, vol. 3, no. 1, pp. 18-31, Jan.-Mar. 2012, doi: 10.1109/T-AFFC.2011.15.

[23] T.-P. Jung, S. Makeig, M. Stensmo, and T. J. Sejnowski, “Estimating alertness from the EEG,” *IEEE Transactions on Biomedical Engineering*, vol. 44, no. 1, pp. 60-69, Jan. 1997, doi: 10.1109/10.553713.

[24] J. G. Webster, Ed., *Medical Instrumentation: Application and Design*, 4th ed. Hoboken, NJ, USA: John Wiley & Sons, 2009.

[25] Texas Instruments, “ADS1299 low-noise, 4-, 6-, 8-channel, 24-bit, analog-to-digital converter for EEG and biopotential measurements,” Dallas, TX, USA, Datasheet SBAS499C, Jul. 2012, rev. Jan. 2017.

[26] U. Rashid, I. K. Niazi, N. Signal, and D. Taylor, “An EEG experimental study evaluating the performance of Texas Instruments ADS1299,” *Sensors*, vol. 18, no. 11, Art. no. 3721, Nov. 2018, doi: 10.3390/s18113721.

[27] EEGLAB, “Extracting data epochs,” *EEGLAB Documentation*, Swartz Center for Computational Neuroscience. [Online]. Available: https://eeglab.org/tutorials/07_Extract_epochs/Extracting_Data_Epochs.html.

[28] The SciPy Community, “scipy.signal.periodogram,” *SciPy Reference Guide*. [Online]. Available: https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.periodogram.html.

[29] The SciPy Community, “scipy.signal.welch,” *SciPy Reference Guide*. [Online]. Available: https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.welch.html.

[30] scikit-learn developers, “Support vector machines,” *scikit-learn User Guide*. [Online]. Available: https://scikit-learn.org/stable/modules/svm.html.

[31] scikit-learn developers, “Ensemble methods: Forests of randomized trees,” *scikit-learn User Guide*. [Online]. Available: https://scikit-learn.org/stable/modules/ensemble.html#random-forests.

[32] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining (KDD ’16)*, San Francisco, CA, USA, Aug. 2016, pp. 785-794, doi: 10.1145/2939672.2939785.

[33] A. Agrawal, J. Gans, and A. Goldfarb, *Power and Prediction: The Disruptive Economics of Artificial Intelligence*. Boston, MA, USA: Harvard Business Review Press, 2022.

[34] R. Bommasani et al., “On the opportunities and risks of foundation models,” arXiv:2108.07258 [cs.LG], Aug. 2021. [Online]. Available: https://arxiv.org/abs/2108.07258.

[35] N. Maslej et al., “The AI Index 2024 Annual Report,” Stanford Institute for Human-Centered Artificial Intelligence, Stanford University, Stanford, CA, USA, Tech. Rep., Apr. 2024.

[36] X. Zhang, L. Yao, X. Wang, J. J. Monaghan, D. McAlpine, and Y. Zhang, “A survey on deep learning-based brain-computer interface: Recent advances and new frontiers,” *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 51, no. 1, pp. 238-262, Jan. 2021, doi: 10.1109/TSMC.2019.2924151.

[37] S. Haegens, H. Cousijn, G. Wallis, P. J. Harrison, and A. C. Nobre, “Inter- and intra-individual variability in alpha peak frequency,” *NeuroImage*, vol. 92, pp. 46-55, 2014, doi:10.1016/j.neuroimage.2014.01.049.

[38] A. S. Geller et al., “Eye closure causes widespread low-frequency power increase and focal gamma attenuation in the human electrocorticogram,” *Clinical Neurophysiology*, vol. 125, no. 9, pp. 1764-1773, Sep. 2014, doi: 10.1016/j.clinph.2014.01.021.

[39] E. M. Whitham et al., “Scalp electrical recording during paralysis: Quantitative evidence that EEG frequencies above 20 Hz are contaminated by EMG,” *Clinical Neurophysiology*, vol. 118, no. 8, pp. 1877-1888, Aug. 2007, doi:10.1016/j.clinph.2007.04.027.

[40] A. Delorme and S. Makeig, “EEGLAB: An open source toolbox for analysis of single-trial EEG dynamics including independent component analysis,” *Journal of Neuroscience Methods*, vol. 134, no. 1, pp. 9-21, Mar. 2004, doi: 10.1016/j.jneumeth.2003.10.009.

[41] R. E. Kalman, “A new approach to linear filtering and prediction problems,” *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35-45, Mar. 1960, doi: 10.1115/1.3662552.

[42] H. E. Rauch, F. Tung, and C. T. Striebel, “Maximum likelihood estimates of linear dynamic systems,” *AIAA Journal*, vol. 3, no. 8, pp. 1445-1450, Aug. 1965, doi: 10.2514/3.3166.

[43] Opracowanie własne autora, zmodyfikowane i udoskonalone estetycznie za pomocą narzędzia ChatGPT Images 2.5 (OpenAI).

[44] scikit-learn developers, “Cross-validation: Evaluating estimator performance,” *scikit-learn User Guide*. [Online]. Available: https://scikit-learn.org/stable/modules/cross_validation.html.

[45] scikit-learn developers, “Metrics and scoring: Quantifying the quality of predictions,” *scikit-learn User Guide*. [Online]. Available: https://scikit-learn.org/stable/modules/model_evaluation.html.

[46] scikit-learn developers, “DummyClassifier,” *scikit-learn API Reference*. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.dummy.DummyClassifier.html.

[47] The SciPy Community, “scipy.signal.butter,” *SciPy Reference Guide*. [Online]. Available: https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.butter.html.

[48] The SciPy Community, “scipy.signal.filtfilt,” *SciPy Reference Guide*. [Online]. Available: https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.filtfilt.html.

[49] B. Efron, “Bootstrap methods: Another look at the jackknife,” *The Annals of Statistics*, vol. 7, no. 1, pp. 1-26, Jan. 1979, doi: 10.1214/aos/1176344552.

[50] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in *Proceedings of the 34th International Conference on Machine Learning*, ser. PMLR, vol. 70, 2017, pp. 1321-1330. [Online]. Available: https://proceedings.mlr.press/v70/guo17a.html.

[51] Neuralink, “A year of Telepathy,” 2025. [Online]. Available: https://neuralink.com/updates/a-year-of-telepathy/.

[52] Reuters, “Musk’s Neuralink gets FDA’s breakthrough device tag for ’Blindsight’ implant,” Sep. 18, 2024. [Online]. Available: https://www.reuters.com/business/healthcare-pharmaceuticals/musks-neuralink-receives-fdas-breakthrough-device-tag-brain-implant-2024-09-17/.

[53] J. Ober, J. Dylak, W. Gryncewicz, and E. Przedpelska-Ober, “Sakkadometria - nowe możliwości oceny stanu czynnościowego ośrodkowego układu nerwowego,” *Nauka*, nr 4, s. 109–135, 2009. [Online]. Available: https://nauka-pan.pl/index.php/nauka/article/view/356.